Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
—
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 2

«Планирование процессов»

по дисциплине «Операционные системы»

Выполнил
студент гр. 5151003/40001 Корпусов Д.Н

<подпись>

Руководитель Югай П.Э

<подпись>

Санкт-Петербург
2025

Содержание

Цель работы 3
Ход работы 3-45
1. Теоретическая часть 3-8
1.1 Потоки и структура struct_thread 3-4
1.2 Планировщик потоков 4-6
1.3 Механизмы синхронизации в PintOS 6-8
2. Практическая часть 8-45
2.1 Реализация приоритетного планирования 8-24
2.2 Реализация механизма разделения приоритета 24-45
Вывод 46
Листинг А «thread.h» 47-49
Листинг Б «thread.c» 50-61
Листинг В «synch.h» 62
Листинг Г «synch.c» 62-70

ЦЕЛЬ РАБОТЫ

Цель работы – изучение механизмов планирования процессов, разработка алгоритма приоритетного планирования и внедрение разработанного алгоритма в учебную операционную систему Pintos.

ХОД РАБОТЫ

1. Теоретическая часть

1.1 Потоки и структура struct_thread.

В операционной системе PintOS каждый выполняемый объект представлен потоком (thread). Для каждого потока выделяется отдельная страница памяти размером 4 КБ. В нижней части страницы хранится структура struct thread, а в верхней — стек ядра, который растёт вниз.

Структура struct thread содержит основную информацию о потоке: идентификатор tid, имя потока (name), текущее состояние (status), приоритет (priority), указатель на вершину стека (stack), элементы для включения в списки (elem, allelem), контрольное значение magic для обнаружения переполнения стека.

Каждый поток может находиться только в одном состоянии из следующих:

THREAD_RUNNING выполняется на процессоре;

THREAD_READY — готов к выполнению, ожидает своей очереди;

THREAD_BLOCKED — заблокирован, ожидает события или ресурса;

THREAD_DYING — завершает работу и подлежит удалению.

Переход между состояниями осуществляется через функции:

Функция	Аргументы	Назначение
thread_block()	нет	Переводит текущий поток в состояние BLOCKED
thread_unblock(struct thread *t)	t- поток, который нужно разблокировать	Возвращает поток в состояние READY
thread_yield()	нет	Текущий поток

		добровольно уступает CPU
schedule()	нет	Выбирает следующий поток для выполнения

Для хранения потоков в различных состояниях PintOS использует двусвязные списки (struct list):

ready_list — очередь готовых к выполнению потоков (READY);

all_list — список всех потоков системы;

waiters — список потоков, ожидающих в семафорах или замках.

Каждый поток имеет поля elem и allelem, которые позволяют включать его в разные списки.

1.2 Планировщик потоков.

Планировщик (scheduler) — это компонент ядра, который управляет выполнением потоков в операционной системе. Он решает, какой поток получит процессор в данный момент времени, и каким образом происходит переключение контекста между потоками.

В исходной версии PintOS планировщик работает по принципу «кто сейчас активен, тот и выполняется». Поток продолжает работу, пока сам не уступит процессор, не заблокируется или не завершится. Очередь готовых потоков (ready_list) организована по порядку поступления: когда текущий поток заканчивает выполнение, планировщик выбирает следующий из начала списка. Приоритеты при этом не учитываются, поэтому все потоки выполняются в порядке очереди, независимо от их важности.

Планировщик работает на основе следующих структур данных:

1. **ready_list** — очередь потоков, готовых к выполнению (состояние THREAD_READY).
2. **all_list** — список всех потоков системы (включая заблокированные и завершающиеся).

3. **idle_thread** — специальный поток, который выполняется, когда нет других готовых потоков. Он выполняет бесконечный цикл ожидания (hlt), чтобы процессор не простаивал впустую.
4. **running_thread()** — макрос, который возвращает указатель на текущий поток, извлекая адрес из стека (так как struct thread лежит в нижней части страницы потока).

Главная функция планировщика — он выбирает поток, который будет выполняться следующим.

```
static void schedule(void) {
  struct thread *cur = running_thread();
  struct thread *next = next_thread_to_run();
  struct thread *prev = NULL;

  ASSERT(intr_get_level() == INTR_OFF);
  ASSERT(cur->status != THREAD_RUNNING);

  if (cur != next)
    prev = switch_threads(cur, next);

  thread_schedule_tail(prev);
}
```

Сохраняется текущий поток (cur), затем вызывается next_thread_to_run(), чтобы выбрать следующий поток, выполняется переключение контекста (switch_threads), завершение действия после переключения (thread_schedule_tail).

Переключение контекста (context switch) — это процесс сохранения состояния текущего потока и восстановления состояния другого потока, выбранного планировщиком для выполнения. В PintOS эта операция выполняется функцией **switch_threads()**, реализованной на ассемблере в файле switch.S.

Когда планировщик принимает решение о смене потока, он вызывает функцию schedule(), которая выбирает следующий поток для выполнения и передаёт управление в switch_threads(cur, next). Функция switch_threads

сохраняет регистры процессора текущего потока (в том числе EBP, EBX, ESI, EDI и EIP), а также указатель стека (ESP) в структуру struct thread текущего потока. Это необходимо для того, чтобы впоследствии можно было продолжить выполнение потока точно с того места, где он был приостановлен.

После сохранения состояния текущего потока функция загружает указатель стека (stack) следующего потока и восстанавливает сохранённые значения регистров, включая адрес возврата (EIP). Затем выполняется инструкция ret, которая передаёт управление на ту точку, где поток был ранее приостановлен или в которую он должен начать выполнение при первом запуске.

1.3 Механизмы синхронизации в PintOS

В многозадачной системе несколько потоков могут обращаться к общим данным или ресурсам. Чтобы избежать повреждения данных при одновременном доступе, в PintOS реализованы специальные механизмы синхронизации: семафоры, замки (locks) и условные переменные (conditions). Они позволяют безопасно организовать взаимодействие между потоками, предотвращая состояния гонки и взаимные блокировки.

Семафор (semaphore) — это счётчик, ограничивающий количество потоков, которым разрешено одновременно пользоваться некоторым ресурсом. Он состоит из двух основных компонентов: целочисленного значения value и списка ожидающих потоков waiters.

Основные операции над семафором:

sema_down() (операция P) — поток проверяет значение value. Если value > 0, оно уменьшается на единицу, и поток продолжает выполнение. Если value == 0, поток блокируется и добавляется в очередь waiters до тех пор, пока ресурс не освободится.

sema_up() (операция V) — увеличивает value на единицу и, если есть ожидающие потоки, будит один из них.

Таким образом, семафор обеспечивает ограниченный доступ к общим ресурсам. Например, если семафор инициализирован значением 1, то одновременно в защищённую область может войти только один поток. Это используется для реализации простых механизмов взаимного исключения.

Замок (lock) — это специальный вид бинарного семафора (value = 1), который может принадлежать только одному потоку в данный момент. Он предназначен для защиты критических секций, где данные должны изменяться строго одним потоком.

Замок реализован поверх семафора и содержит дополнительное поле holder, указывающее на поток-владельца. Только поток, захвативший замок с помощью lock_acquire(), имеет право освободить его через lock_release().

Алгоритм работы замка следующий:

1. Поток вызывает lock_acquire() — выполняется sema_down(). Если замок занят, поток блокируется.
2. После получения замка поле holder заполняется текущим потоком.
3. При вызове lock_release() замок освобождается (sema_up()), и один из ожидающих потоков разблокируется.

Главное отличие замка от обычного семафора состоит в том, что только владелец замка может его освободить, что предотвращает ошибки синхронизации. Кроме того, в PintOS замки не являются рекурсивными — попытка повторно захватить уже удерживаемый замок вызывает ошибку выполнения.

Условная переменная (condition) используется для организации взаимодействия между потоками, ожидающими наступления некоторого события. Каждая условная переменная связана с определённым замком и позволяет потоку временно освободить замок и перейти в состояние ожидания.

Работа с условными переменными выполняется с помощью трёх операций:

cond_wait() — поток освобождает замок, засыпает (вызов sema_down()), и снова захватывает замок при пробуждении;

cond_signal() — пробуждает один поток, ожидающий данного условия;

cond_broadcast() — пробуждает все потоки, ожидающие условия.

Когда поток вызывает cond_wait(), он помещается в очередь ожидания переменной и блокируется. Другой поток, вызвавший cond_signal(), пробуждает одного из спящих потоков, позволяя ему продолжить выполнение после повторного захвата замка.

Все механизмы синхронизации в PintOS тесно связаны с планировщиком потоков. Когда поток вызывает операции sema_down() или lock_acquire() и ресурс недоступен, он переводится в состояние **BLOCKED** с помощью thread_block(). Когда другой поток освобождает ресурс (sema_up() или lock_release()), ожидающий поток переводится обратно в состояние **READY** (thread_unblock()) и добавляется в очередь готовых к выполнению потоков.

Таким образом, планировщик и система синхронизации совместно обеспечивают безопасное и справедливое выполнение потоков, предотвращая одновременный доступ к критическим ресурсам и обеспечивая корректное пробуждение ожидающих потоков.

2. Практическая часть

2.1 Реализация приоритетного планирования

В ходе лабораторной работы была исследована работа планировщика потоков в операционной системе Pintos. Была построена диаграмма исполнения процессов в исходном алгоритме без учёта приоритетов и разделения приоритетов.

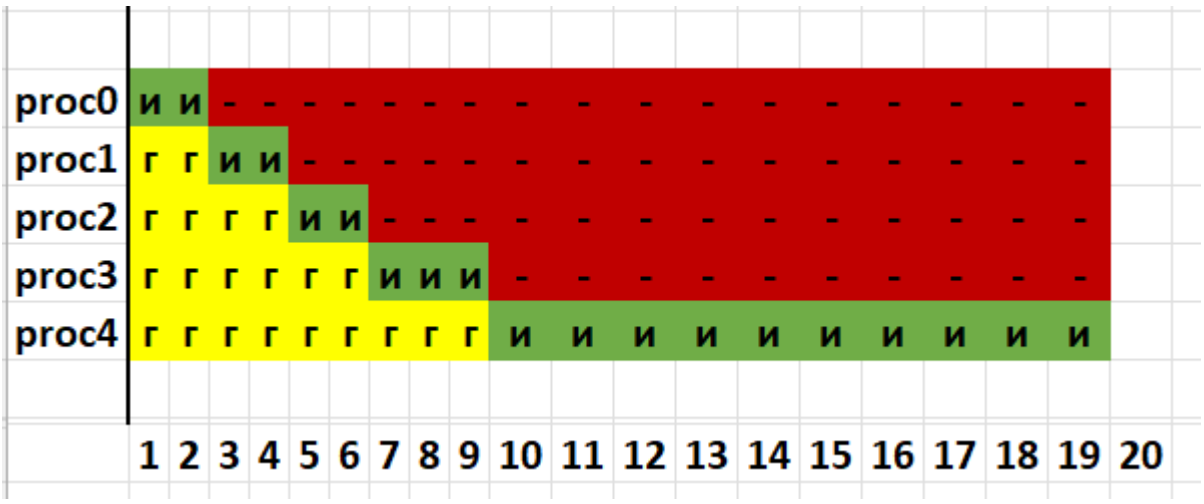


Рис. 1 «Диаграмма исполнения процессов для изначального алгоритма Pintos-a»

Для исходного алгоритма планирования в системе **Pintos** были рассчитаны основные временные показатели на основе диаграммы исполнения процессов (рис. 1).

Сначала было подсчитано количество тактов, в течение которых процессы находились в состоянии ожидания — символ «г».

Среднее время ожидания вычисляется по формуле:

Tср. вр ожид = Σ(г) / N = 21 / 5 = 4,2 кванта

Далее было подсчитано суммарное время выполнения каждого процесса, включающее периоды ожидания и исполнения («г» + «и»). Тогда полное среднее время исполнения:

Tполн ср исполн = Σ(г + и) / N = (21+19) / 5 = 8 кванта

Также была составлена блок-схема изначального алгоритма планирования.



Рис. 2 «Блок-схема изначального алгоритма планирования»

В ходе лабораторной работы были проанализированы и изменены исходные файлы ядра PintOS: thread.c, thread.h, synch.c, synch.h. Цель — реализовать приоритетное планирование потоков. Для проверки работы использовались тесты из каталога tests/threads.

1) **Первый тест – alarm-priority**.

Тест **alarm-priority** проверяет, что после пробуждения потоков таймером первыми выполняются потоки с более высоким приоритетом. В начале задаётся общее время пробуждения — все десять потоков засыпают до одного и того же момента. Каждый поток создаётся с разным приоритетом и уходит в сон через timer_sleep(). Когда время истекает, будильник пробуждает их одновременно, и планировщик должен выбрать для выполнения поток с наибольшим приоритетом. По мере пробуждения потоки выводят сообщение о своём имени, что позволяет увидеть порядок их запуска. Если планировщик реализован правильно, сообщения появляются в порядке убывания приоритета. Таким образом, тест показывает, что система учитывает приоритеты при переходе потоков из состояния BLOCKED в READY.

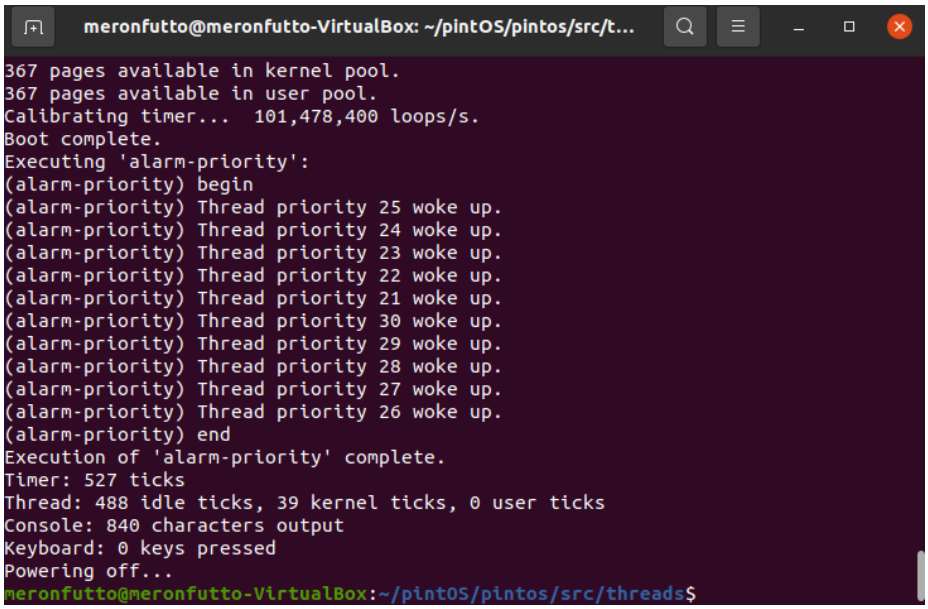


Рис.3 «alarm-priority тест (старый алгоритм)»

На рисунке показан результат выполнения теста **alarm-priority** до внедрения приоритетного планирования. Видно, что потоки пробуждаются не в порядке убывания приоритета, как требуется по условию теста, а в случайной последовательности: сначала выводятся потоки с низкими приоритетами (25, 24, 23…), а потоки с более высокими приоритетами (30, 29, 28…) появляются только позже. Это подтверждает, что исходный

планировщик PintOS не учитывает значение приоритета при выборе потока для выполнения — все потоки обрабатываются по порядку поступления, без различия по важности.

Изучив работу теста **alarm-priority**, стало ясно, что некорректный порядок пробуждения потоков связан с тем, что планировщик PintOS не учитывает их приоритет при добавлении в очередь готовых. Для исправления этого поведения были определены функции, требующие изменений. В функции thread_unblock() необходимо реализовать вставку потоков в очередь ready_list в порядке убывания приоритета, чтобы более важные потоки выполнялись первыми. Также нужно добавить проверку, чтобы текущий поток уступал процессор, если в системе появился поток с более высоким приоритетом. В thread_create() — реализовать аналогичную проверку сразу после создания нового потока. Эти изменения позволили планировщику корректно обрабатывать приоритеты и обеспечить правильный порядок выполнения потоков.

```
void
thread_unblock (struct thread *t)
{
  enum intr_level old_level;

  ASSERT (is_thread (t));

  old_level = intr_disable ();
  ASSERT (t->status == THREAD_BLOCKED);
  list_insert_ordered(&ready_list,                    &t->elem,
comparator_greater_thread_priority, NULL); // Вставляем в ready-list
по убыванию приоритета
  t->status = THREAD_READY;

  if (thread_current() != idle_thread && thread_current()-
>priority < t->priority)   // Если текущий процесс - не холостой и его
приоритет ниже чем у разблокированного, вызываем планировщик
  {
    thread_yield();
  }
  intr_set_level (old_level);
}

void
thread_yield (void)
{
  struct thread *cur = thread_current ();
  enum intr_level old_level;
```

```
        ASSERT (!intr_context ());

        old_level = intr_disable ();
        if (cur != idle_thread){
          list_insert_ordered(&ready_list,&cur->elem,
comparator_greater_thread_priority,  NULL);  //  Также  текущий  поток
вставляем в нужное место в список ready
        }
        cur->status = THREAD_READY;
        schedule();
        intr_set_level (old_level);
      }
```

В функции **thread_unblock()** были внесены изменения, чтобы учесть приоритет потоков при их пробуждении. Ранее поток, переходящий из состояния BLOCKED в READY, просто добавлялся в конец очереди ready_list, из-за чего порядок выполнения не зависел от приоритета. Теперь вместо этого используется функция list_insert_ordered() (из list.c), которая вставляет поток в очередь в порядке убывания приоритета с помощью написанного компаратора comparator_greater_thread_priority. Благодаря этому потоки с более высоким приоритетом оказываются в начале списка и получают процессор первыми. Кроме того, добавлена проверка: если текущий поток не является idle_thread и его приоритет ниже, чем у разблокированного потока, вызывается thread_yield(), чтобы немедленно уступить процессор более приоритетному потоку. Эти изменения обеспечили корректный порядок планирования при пробуждении потоков.

После исправления алгоритма повторно запустили тест.

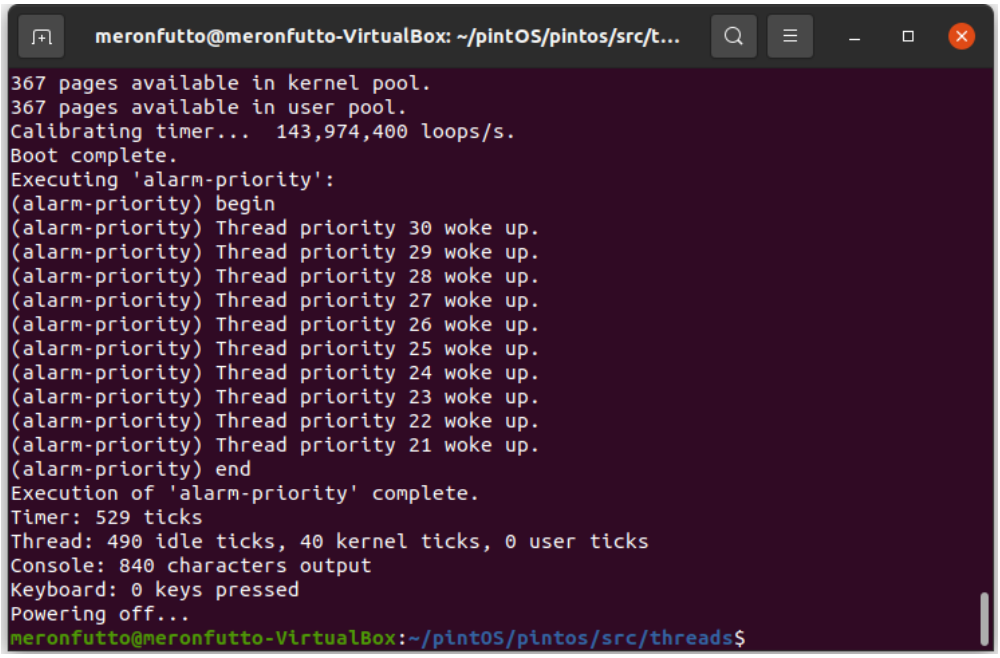


Рис.2 «alarm-priority тест (новый алгоритм)»

После внесённых изменений тест **alarm-priority** завершился успешно: потоки пробуждаются строго в порядке убывания приоритета — сначала с наибольшим (30), затем с меньшими. Это подтверждает, что модифицированная функция thread_unblock() корректно сортирует очередь готовых потоков и планировщик теперь выбирает для выполнения поток с наивысшим приоритетом.

2) **Тест priority-change**.

Тест **priority-change** проверяет, как система реагирует на изменение приоритета у потока, который в данный момент выполняется. По условию, если выполняющийся поток понижает свой приоритет и в системе есть другой поток с более высоким приоритетом, он должен немедленно уступить процессор новому потоку.

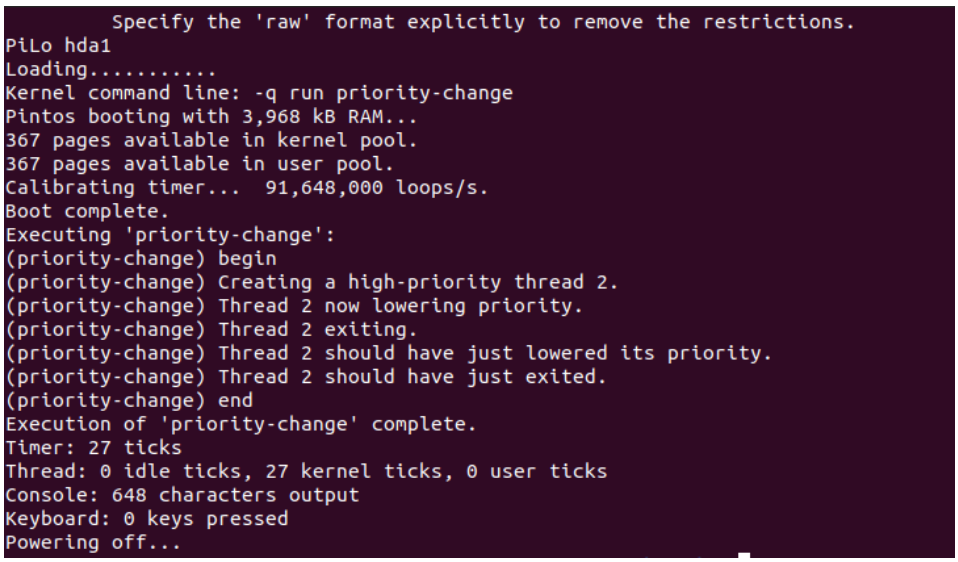


Рис. 3 «Тест priority-change до исправления алгоритма»

Тест **priority-change** проверяет, что при понижении приоритета выполняющийся поток уступает процессор потоку с более высоким приоритетом(тот, что печатает should have…). На экране видно, что после понижения приоритета поток 2 продолжает выполняться, хотя в системе уже существует поток с более высоким приоритетом. Это означает, что планировщик не сделал переключение контекста, как ожидалось.

После анализа теста **priority-change** стало ясно, что планировщик не уступает процессор при понижении приоритета текущего потока. Для исправления этого поведения были внесены изменения в функцию thread_set_priority().

```
void
thread_set_priority (int new_priority)
{
  struct thread* t_current = thread_current();
  if( t_current->priority == t_current->original_priority)
  {
    t_current->priority = new_priority;
    t_current->original_priority = new_priority;
  }
  else
  {
    t_current->original_priority=new_priority;
  }


  if (!list_empty(&ready_list))    //Если список готовых потоков
не пустой
  {
```

```
          struct                                                    thread*
next=list_entry(list_begin(&ready_list),struct  thread,elem);  //  Берём
первый элемент из ready-лист(он с самым высоким приоритетом)
          if(next   !=NULL   &&   next->priority   >   new_priority)   //
Проверяем  что  поток  существует  и  что  его  приоритет  больше  чем  у
текущего потока
          {
            thread_yield(); //Блокируем текущий поток
          }
        }
      }
```

После установки нового значения приоритета добавлена проверка: если в очереди готовых потоков (ready_list) присутствует поток с более высоким приоритетом, текущий поток немедленно вызывает thread_yield() и уступает процессор. Это гарантирует правильное переключение контекста при изменении приоритета и обеспечивает прохождение теста **priority-change**.

Повторно был проведён тест.

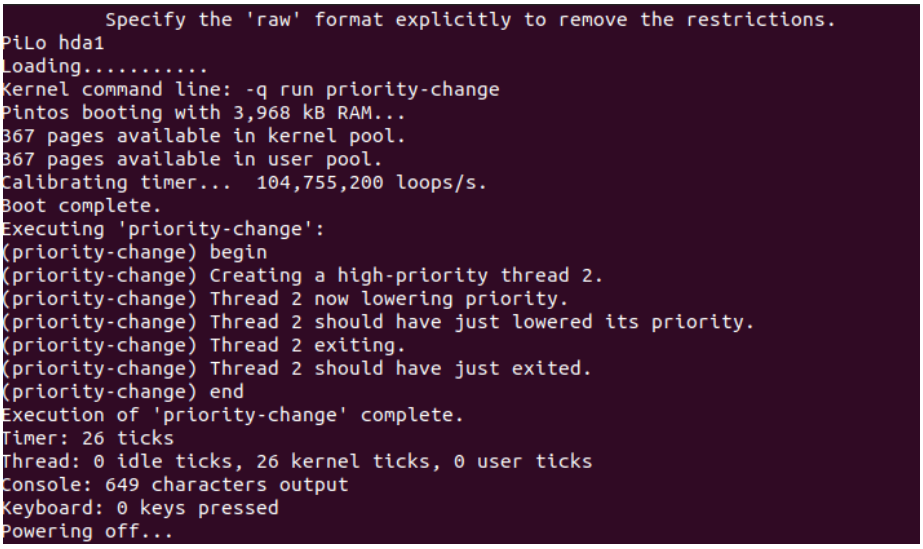


Рис. 4 «Тест priority-change (новый алгоритм)»

После внесения изменений тест **priority-change** завершился корректно. Поток с пониженным приоритетом сразу уступает процессор потоку с более высоким приоритетом, что подтверждает правильную работу планировщика. Теперь система реагирует на изменение приоритета динамически — если текущий поток теряет статус самого приоритетного, он немедленно передаёт управление.

3) Тест priority-fifo

Тест priority-fifo проверяет, что в системе планирования потоков с одинаковым приоритетом выполняется строгое чередование по принципу "first in, first out" (FIFO), то есть справедливый круговой порядок (round-robin).

Создаются 16 потоков (все с одинаковым приоритетом). Каждый поток выполняет простую функцию — несколько раз захватывает общий lock, записывает свой номер (id) в общий буфер output, отпускает lock и вызывает thread_yield(), уступая процессор другим. Таким образом, порядок выполнения потоков напрямую зависит от планировщика.

```
Kernel command line: -q run priority-fifo
Pintos booting with 3,968 kB RAM...
367 pages available in kernel pool.
367 pages available in user pool.
Calibrating timer...  166,912,000 loops/s.
Boot complete.
Executing 'priority-fifo':
(priority-fifo) begin
(priority-fifo) 16 threads will iterate 16 times in the same order each time.
(priority-fifo) If the order varies then there is a bug.
(priority-fifo) iteration: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(priority-fifo) iteration: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(priority-fifo) iteration: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(priority-fifo) iteration: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(priority-fifo) iteration: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(priority-fifo) iteration: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(priority-fifo) iteration: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(priority-fifo) iteration: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(priority-fifo) iteration: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(priority-fifo) iteration: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(priority-fifo) iteration: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(priority-fifo) iteration: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(priority-fifo) iteration: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(priority-fifo) iteration: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(priority-fifo) iteration: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(priority-fifo) iteration: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(priority-fifo) end
Execution of 'priority-fifo' complete.
Timer: 43 ticks
Thread: 0 idle ticks, 44 kernel ticks, 0 user ticks
Console: 1557 characters output
Keyboard: 0 keys pressed
Powering off...
```

Рис. 5 «priority-fifo тест»

Тест priority-fifo успешно проходит на той же реализации планировщика, которая была разработана ранее для тестов alarm-priority и priority-change. Это означает, что все необходимые механизмы уже

реализованы: сортировка ready_list по приоритету, корректная вставка потоков в список готовых к выполнению и сохранение порядка выполнения (round-robin) для потоков с одинаковым приоритетом.

4) Тест priority-preempt

Тест проверяет, что поток с более высоким приоритетом действительно вытесняет текущий и остаётся лидером до завершения.

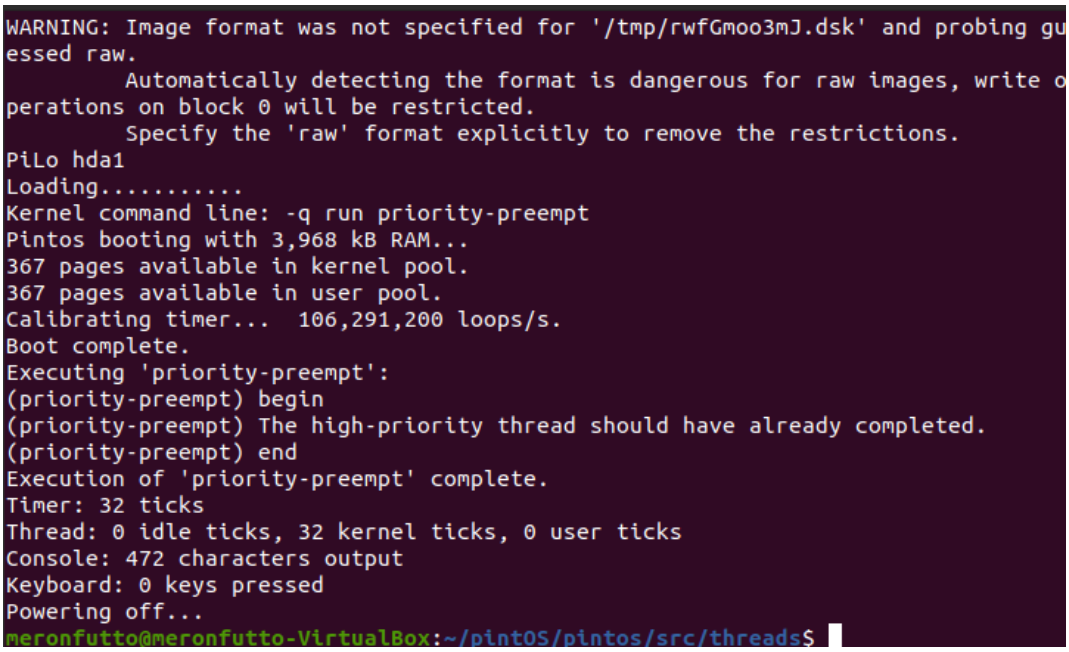


Рис. 6 «priority-preempt до внесённых изменений»

До внесённых изменений главный поток не уступал процессору более приоритетному потоку, поэтому тот не выполнялся вовсе.

После реализации упорядоченной вставки в ready_list и вызова thread_yield() при появлении потока с более высоким приоритетом вытеснение стало происходить корректно. Теперь тест показывает, что высокоприоритетный поток выполняется полностью, а главный получает управление только после его завершения.

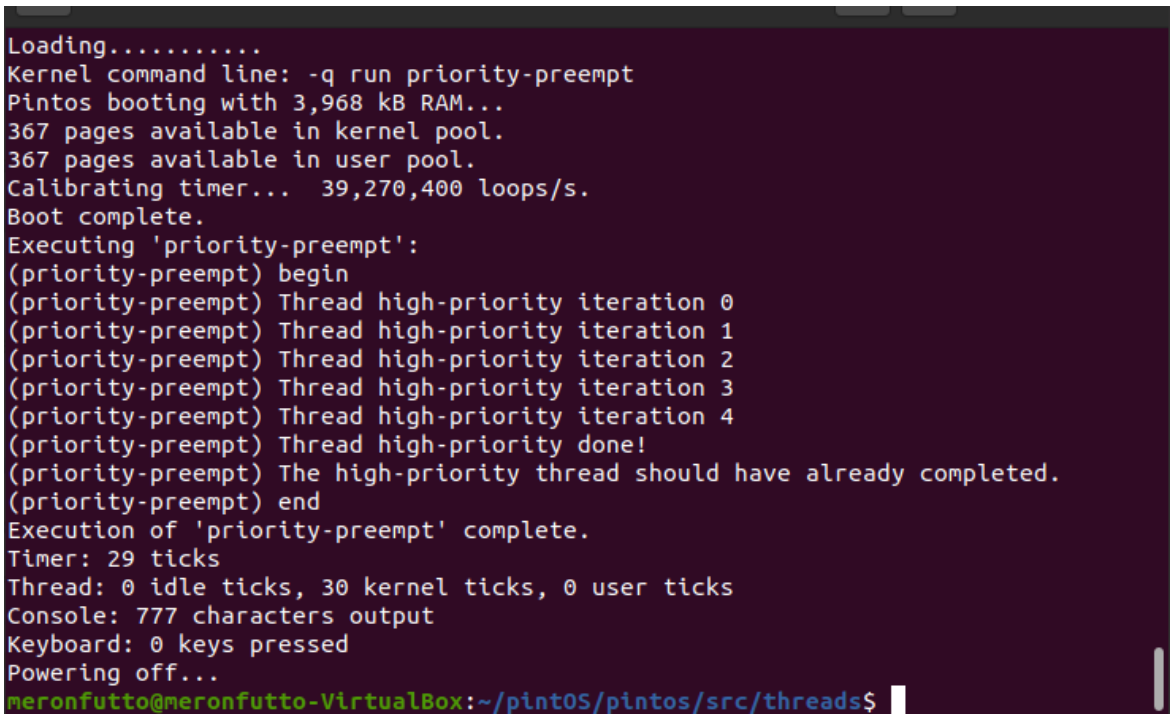


Рис. 7 «Тест priority-preempt после внесённых изменений»

5) Тест priority-sema

Суть теста priority-sema заключается в проверке того, что при работе семафора в первую очередь пробуждается поток с самым высоким приоритетом из ожидающих. Главный поток создаёт несколько потоков с разными приоритетами, которые блокируются на sema_down(), а затем поочерёдно вызывает sema_up(). Ожидается, что каждый раз sema_up() разблокирует не первый в очереди, а наиболее приоритетный поток, который сразу же вытесняет основной поток и выполняется первым.

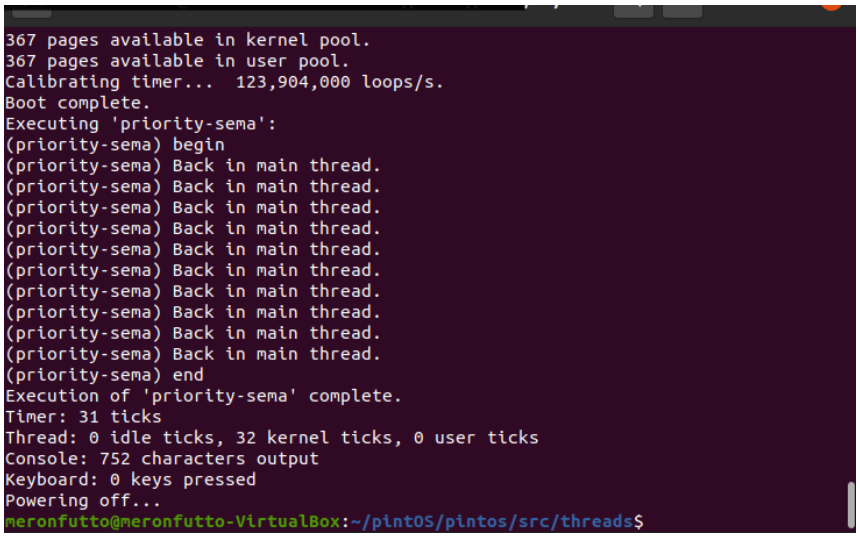


Рис. 8 «Тест priority-preempt до внесения изменений»

В ходе анализа теста priority-sema было замечено, что при выполнении команды sema_up() высокоприоритетные потоки не выполнялись сразу, а выводился только текст от основного потока Back in main thread. Это означало, что потоки хоть и разблокировались, но не получали процессорное время, то есть механизм вытеснения не срабатывал. Причина заключалась в том, что функции sema_down() и sema_up() не учитывали приоритет потоков: при ожидании потоки вставлялись в список waiters без сортировки, а при пробуждении sema_up() просто разблокировала первый попавшийся поток, не обязательно с наибольшим приоритетом.

```
void
sema_down (struct semaphore *sema)
{
  enum intr_level old_level;

  ASSERT (sema != NULL);
  ASSERT (!intr_context ());

  old_level = intr_disable ();
  while (sema->value == 0)
    {
      list_insert_ordered(&sema->waiters,       &thread_current()-
>elem,   comparator_greater_thread_priority,   NULL);   /*   вставка   по
убыванию приоритета в лист ожидающих */
      thread_block ();
    }
  sema->value--;
  intr_set_level (old_level);
}
```

Для исправления этого алгоритма в sema_down() была добавлена упорядоченная вставка потока в очередь ожидания по убыванию приоритета с помощью list_insert_ordered(). В sema_up() было сделано так, чтобы разблокировался первый элемент очереди (самый приоритетный поток), и после этого происходила немедленная остановка текущего потока, если приоритет разбуженного выше.

После этих изменений тест стал проходить: при каждом sema_up() первым выполняется поток с наибольшим приоритетом, выводится строка Thread ... woke up., и только затем продолжается основной поток.

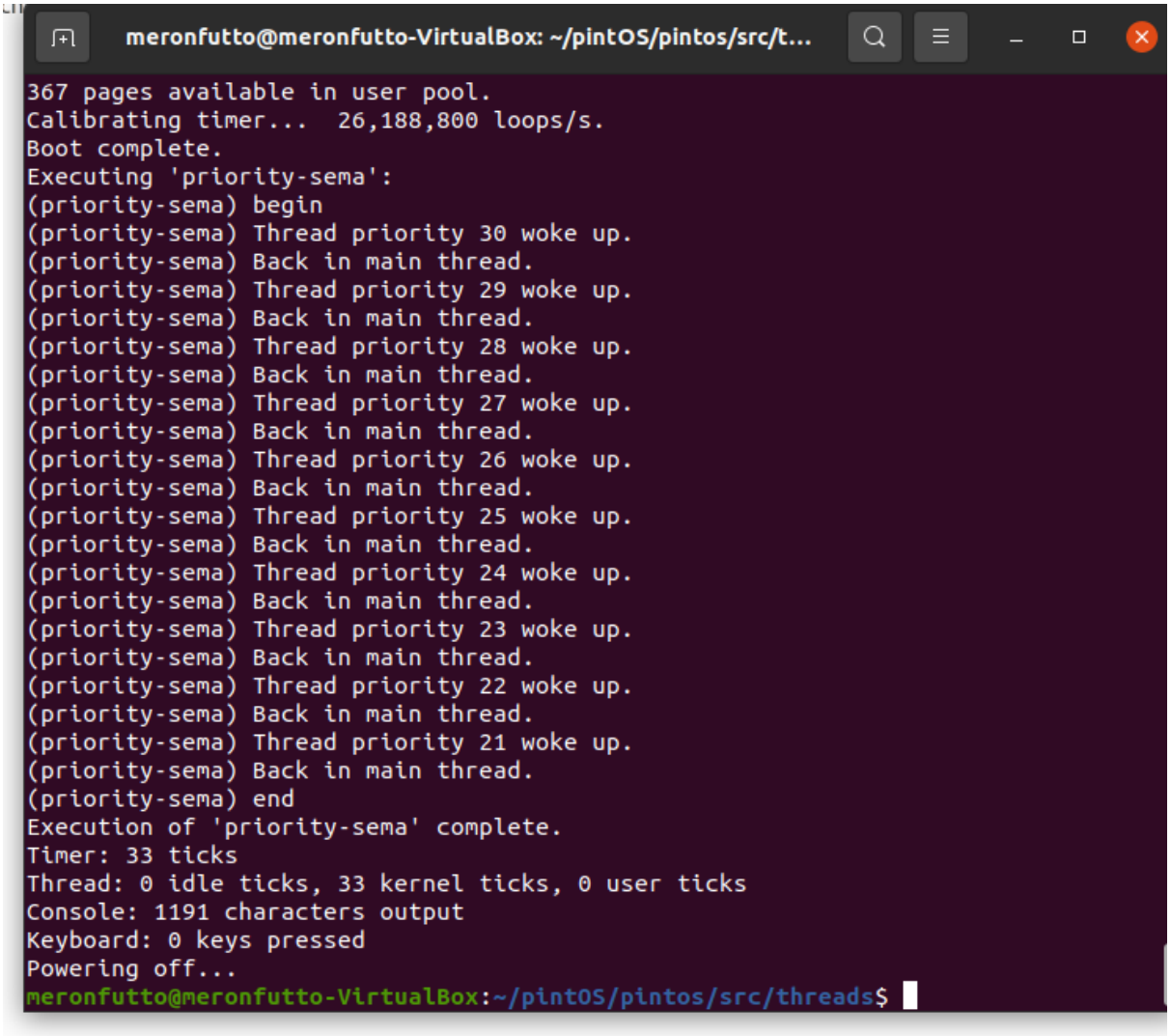


Рис. 9 «Тест priority-sema после внесённых изменений»

6) Тест priority-condvar

Суть теста priority-condvar — проверить, что функция cond_signal() будит поток с наивысшим приоритетом, который ждал на условной переменной (condition), а не просто первый попавшийся.

Главный поток создаёт десять потоков с разными приоритетами. Каждый поток захватывает замок и вызывает cond_wait(), где он отпускает замок и засыпает, ожидая сигнала. После этого главный поток по очереди десять раз захватывает замок, вызывает cond_signal() и освобождает замок. Каждый сигнал будит одного ждущего потока, причём должен проснуться именно самый приоритетный. Разбуженный поток снова захватывает замок, выводит сообщение и отпускает его. Таким образом, проверяется, что

очередь ожидания условной переменной упорядочена по приоритету и пробуждение происходит в правильном порядке.

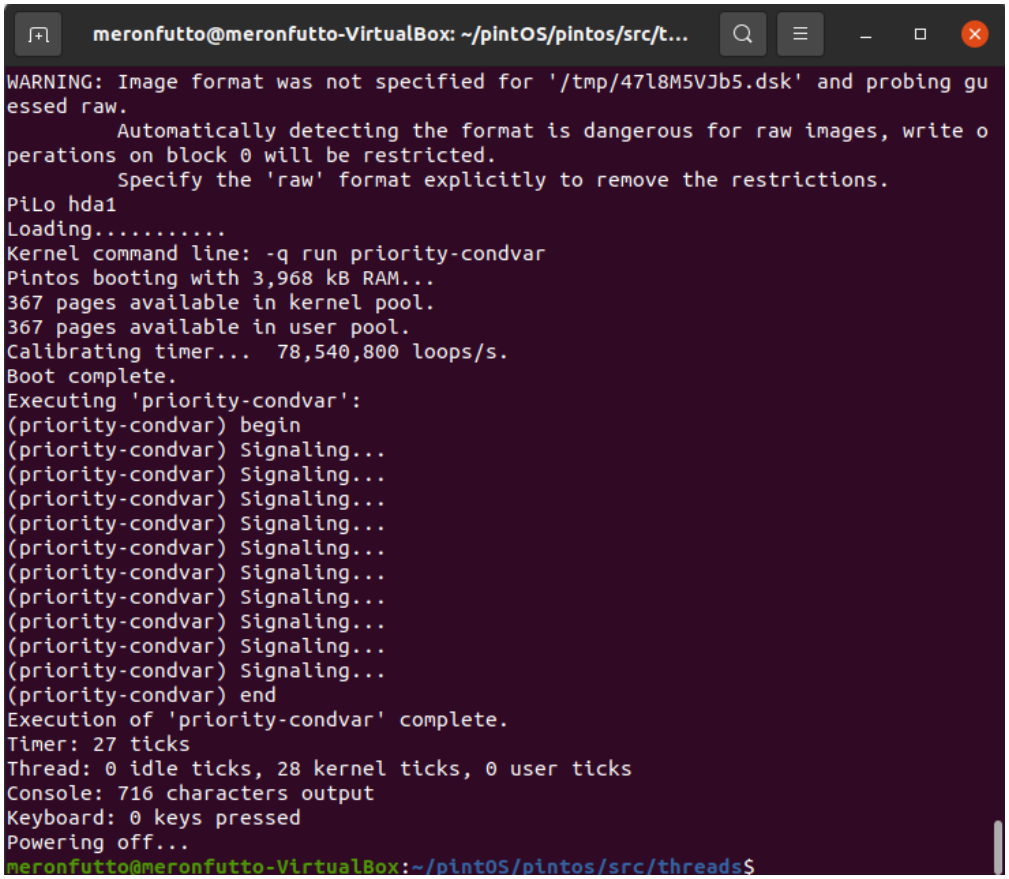


Рис. 10 «Тест priority-condvar до внесения изменений»

До внесения изменений тест priority-condvar показывает, что главный поток печатает только строки "Signaling...", а сообщения от остальных потоков ("Thread ... woke up.") не появляются вовсе. Это значит, что потоки, ожидавшие сигнала, не получают процессорное время сразу после пробуждения — то есть механизм вытеснения не срабатывает. Главный поток успевает последовательно вызвать cond_signal() десять раз и завершить выполнение, пока разбуженные потоки остаются в очереди готовых.

После анализа теста было выявлено, что потоки просыпались не в порядке приоритетов, потому что в функции cond_wait() они добавлялись в список ожидания без сортировки. Из-за этого cond_signal() пробуждал первый попавшийся поток, а не самый приоритетный.

```
void
cond_wait (struct condition *cond, struct lock *lock)
{
  struct semaphore_elem waiter;

  ASSERT (cond != NULL);
  ASSERT (lock != NULL);
  ASSERT (!intr_context ());
  ASSERT (lock_held_by_current_thread (lock));

  sema_init (&waiter.semaphore, 0);

  waiter.semaphore.priority = thread_current()->priority;

  list_insert_ordered(&cond->waiters, &(waiter.elem), comparator_greater_sema_priority, NULL); // вставка в лист ожидания условия по убыванию приоритета

  lock_release (lock);
  sema_down (&waiter.semaphore);
  lock_acquire (lock);
}
```

Чтобы исправить это, в cond_wait() изменили добавление элемента в список — теперь поток вставляется упорядоченно по убыванию приоритета с помощью list_insert_ordered(&cond->waiters, &(waiter.elem), comparator_greater_sema_priority, NULL). Это гарантирует, что при вызове cond_signal() из списка ожидания будет разбужен именно поток с наибольшим приоритетом. После этого тест priority-condvar стал проходить: теперь просыпаются потоки в правильном порядке — от более приоритетных к менее приоритетным.

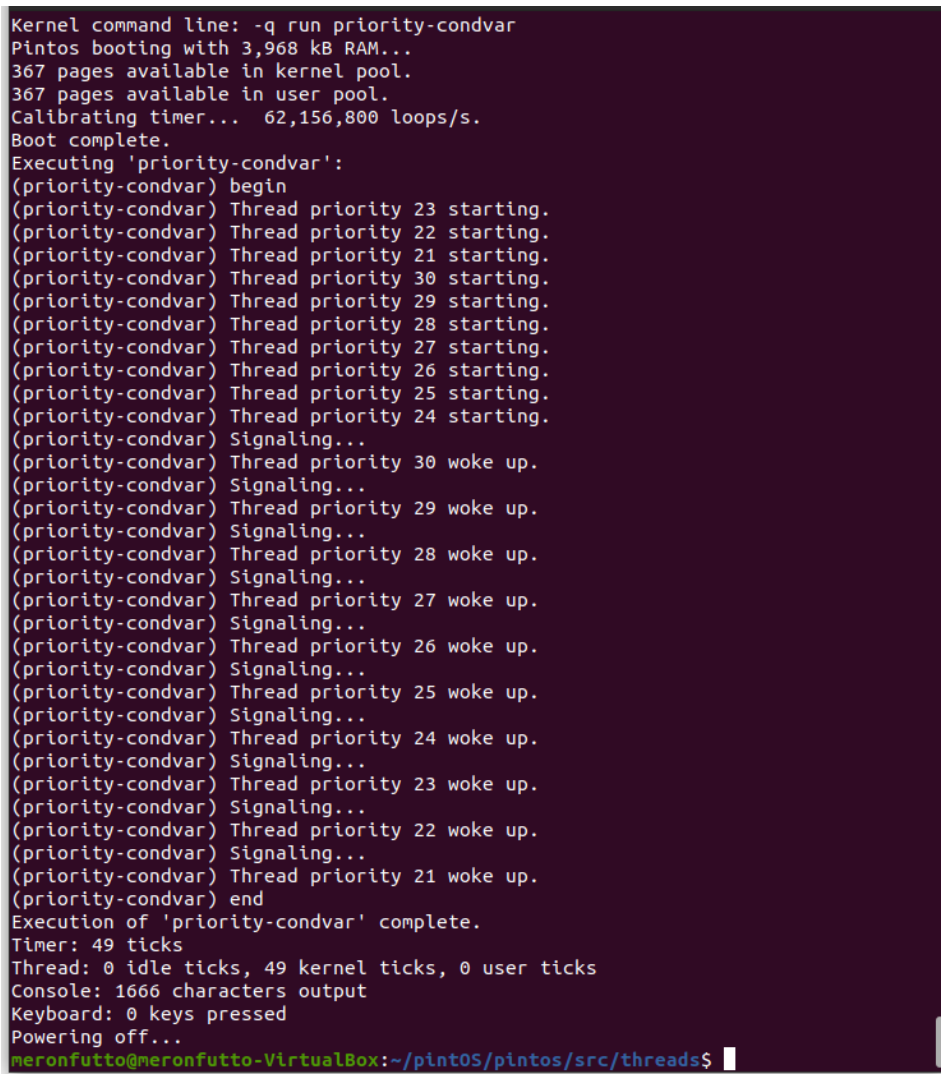


Рис. 11 «Тест priority-condvar после изменений»

2.2 Реализация механизма разделения приоритета

В стандартной реализации PintOS приоритетный планировщик работает без механизма наследования приоритета, из-за чего возникает проблема инверсии приоритетов. Это происходит, когда поток с высоким приоритетом блокируется, ожидая замок, который удерживает поток с более низким приоритетом. Пока низкоприоритетный поток не завершит работу, высокоприоритетный остаётся в ожидании, а процессор может выполнять другие, менее важные задачи. В такой ситуации приоритетное планирование теряет смысл: система перестаёт гарантировать выполнение действительно важных потоков первыми.

Основная причина этого в том, что в PintOS приоритеты сравниваются только среди потоков в списке готовых (ready_list), но не передаются между

потоками, связанными зависимостями через замки и семафоры. В результате поток, удерживающий ресурс, может замедлить выполнение более приоритетных потоков, которые от него зависят. Исправляется это введением механизма **разделения приоритета**, который позволяет временному повышению приоритета владельца замка до уровня потока, ожидающего этот ресурс.

Чтобы исправить это, были внесены изменения в функции lock_acquire() и lock_release() в файле **synch.c**, а также добавлены необходимые поля в структуру потока (waiting_lock, locks, original_priority) в **thread.h**.

```
       struct thread
         {
           /* Owned by thread.c. */
           tid_t tid;                          /* Thread identifier. */
           enum thread_status status;          /* Thread state. */
           char name[16];                        /* Name (for debugging
purposes). */
           uint8_t *stack;                      /* Saved stack pointer.
*/
           int priority;                       /* Priority. */
           int original_priority;                  /* priority, before
donation */

           struct list_elem allelem;              /* Узел двусвязного
списка, для глоб-ого сп-ка прц-ов */


           /* Shared between thread.c and synch.c. */
           struct list_elem elem;                /* Узел двусв. списка
для динам. очередей(ready лист) */

           // need for priority donations
           struct  lock*  waiting_lock;    //  объект  блокировки,к-ый
ожидает этот поток
           struct list locks; //list of locks the thread holds

       #ifdef USERPROG
           /* Owned by userprog/process.c. */
           uint32_t *pagedir;                  /* Page directory. */
       #endif

           /* Owned by thread.c. */
           unsigned magic;                          /* Detects stack
overflow. */
         };
```

```
      void
      lock_acquire (struct lock *lock)
      {
        ASSERT (lock != NULL);
        ASSERT (!intr_context ());
        ASSERT (!lock_held_by_current_thread (lock));

        // priority donation, when locking
        struct lock* current_lock=lock;
        struct  thread*  t_holder  =  lock->holder;  //  current  holder
thread
        struct thread* t_current = thread_current();

        // the current process is waiting on [lock]
        t_current->waiting_lock=lock;

        if(t_holder == NULL)
        {
          current_lock->priority = t_current->priority;
        }

        while(t_holder  !=NULL  &&  t_holder->priority  <  t_current-
>priority)
        {
          //donate priority to [t_holder]
          thread_priority_donate(t_holder,t_current->priority);

          if (current_lock->priority < t_current->priority)
          {
            current_lock->priority = t_current -> priority;
          }

          current_lock = t_holder->waiting_lock;
          if (current_lock == NULL) break;
          t_holder = current_lock->holder;


        }

        sema_down (&lock->semaphore);
        lock->holder = thread_current ();

        lock->holder->waiting_lock = NULL; // no longer waiting
        list_insert_ordered(&(lock->holder->locks), &(lock->lockelem),
comparator_greater_lock_priority, NULL);
      }

      void
      thread_set_priority  (int  new_priority)                        //
**********************
      {
        struct thread* t_current = thread_current();
        if( t_current->priority == t_current->original_priority)
        {
          t_current->priority = new_priority;
          t_current->original_priority = new_priority;
```

```
        }
        else
        {
          t_current->original_priority=new_priority;
        }


        if (!list_empty(&ready_list))
        {
          struct                                                    thread*
next=list_entry(list_begin(&ready_list),struct thread,elem);
          if(next !=NULL && next->priority > new_priority)
          {
            thread_yield();
          }
        }
      }
```

При вызове lock_acquire() поток пытается захватить замок. Если замок свободен, поток просто получает его и продолжает выполнение. Если же замок уже удерживается другим потоком, запускается механизм наследования приоритета (priority donation). В этом случае текущий поток сохраняет указатель на замок, которого он ожидает, в поле waiting_lock, чтобы можно было отследить зависимость. Далее проверяется, есть ли у замка владелец. Если он существует и его приоритет ниже, текущий поток передаёт ему свой приоритет. Эта передача выполняется рекурсивно — если владелец замка также ждёт другой замок, приоритет распространяется дальше по цепочке, пока не достигнет конца зависимости. Благодаря этому приоритет самого важного потока временно “поднимает” все потоки, блокирующие его выполнение. После получения доступа к замку выполняется sema_down(), поток становится его владельцем, очищает поле waiting_lock и добавляет захваченный замок в свой список locks. Список удерживаемых замков упорядочивается по приоритету, чтобы при освобождении можно было корректно восстановить исходный приоритет. Таким образом, lock_acquire() гарантирует, что потоки с высоким приоритетом не будут “застревать” из-за низкоприоритетных владельцев.

```
      void
      lock_release (struct lock *lock)
      {
        ASSERT (lock != NULL);
        ASSERT (lock_held_by_current_thread (lock));
```

```
        struct thread* t_current = thread_current();

        lock->holder = NULL;
        sema_up (&lock->semaphore);

        // Remove the lock from locklist
        list_remove(&lock->lockelem);

        // priority donation: restoration
        if(list_empty(&t_current->locks))
        {
          // no more locks: there is no priority donors
          // the original priority of the current thread
          thread_priority_donate(t_current,                t_current-
>original_priority);
        }
        else
        {
          // donated: lookup the donors, find the highest priority
lock
          // then it should be the (newly updated) donated priority of
t
          list_sort(&(t_current-
>locks),comparator_greater_lock_priority,NULL);
          struct           lock*           highest_lock           =
list_entry(list_front(&(t_current->locks)),struct lock, lockelem);
          thread_priority_donate(t_current, highest_lock->priority);
        }
      }
```

Функция lock_release() освобождает замок и восстанавливает приоритет потока, если он ранее был повышен из-за donation.

Сначала проверяется, что замок действительно принадлежит текущему потоку. Затем у замка сбрасывается владелец (lock->holder = NULL), и вызывается sema_up(), чтобы разбудить первый поток из очереди ожидания — тот, кто теперь сможет захватить замок. После этого текущий замок удаляется из списка locks, который хранит все замки, удерживаемые данным потоком.

Далее идёт ключевая часть — восстановление приоритета. Если поток больше не удерживает ни одного замка, значит, ему никто не пожертвовал приоритет, и можно вернуть исходное значение (original_priority). Если же замки остались, то нужно найти среди них тот, у которого наивысший приоритет ожидания. Для этого список locks сортируется по приоритету, и текущему потоку назначается приоритет, равный приоритету самого

“важного” замка (т.е. того, чей ожидающий поток имеет наибольший приоритет).

Таким образом, lock_release() не только освобождает ресурс, но и корректно возвращает потоку его собственный приоритет или оставляет временно повышенный, если donation ещё действует.

1) Тест priority-donate-one

Тест priority-donate-one проверяет работу механизма разделения приоритета при захвате замка. Главный поток берёт замок и создаёт два потока с более высоким приоритетом. Эти потоки пытаются захватить тот же замок, но блокируются, потому что он уже занят главным потоком. В этот момент происходит передача приоритета — сначала главный поток получает приоритет первого ожидающего потока, затем второго, если у него приоритет выше. Таким образом, приоритет главного временно повышается, чтобы он как можно быстрее освободил замок и не мешал более приоритетным потокам. После освобождения замка приоритет возвращается к исходному, и потоки acquire2 и acquire1 выполняются в порядке своих приоритетов. Тест подтверждает, что приоритет корректно передаётся владельцу замка и снимается после его освобождения.

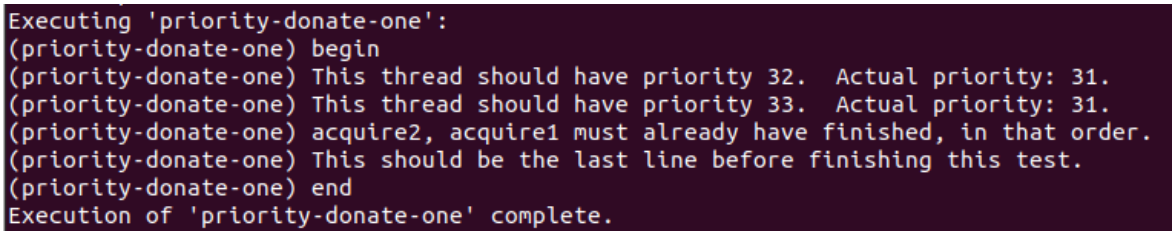


Рис. 12 «Тест priority-donate-one до внесённых изменений»

До внесения изменений тест priority-donate-one показал, что механизм наследования приоритета в системе не работает. Главный поток, удерживающий замок, не получил повышение приоритета от ожидающих потоков с более высоким приоритетом. В выводе видно, что фактический приоритет главного потока остаётся равным 31, хотя должен был временно повыситься до 32 и 33. Это означает, что при блокировке потоков на замке их приоритеты не передаются владельцу ресурса, из-за чего

высокоприоритетные потоки не могут выполнить свою работу первыми. Таким образом, тест демонстрирует наличие проблемы инверсии приоритетов и необходимость реализации механизма наследования приоритета.

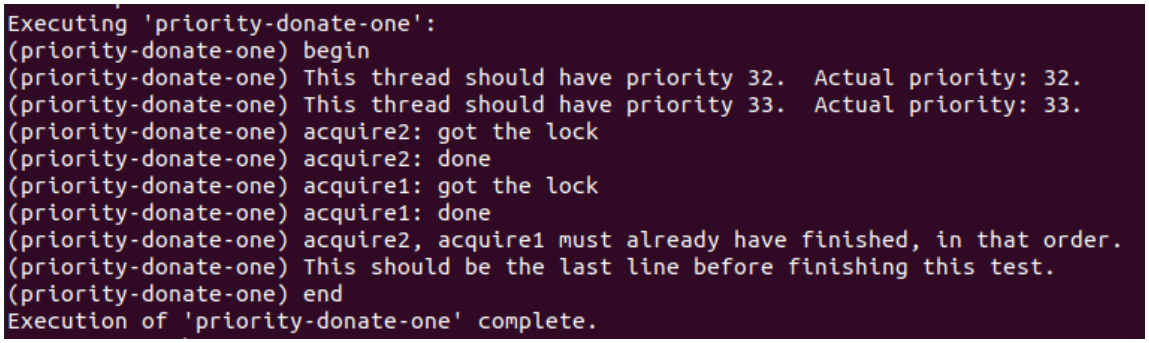


Рис. 12 «Тест priority-donate-one после внесённых изменений»

После внесения изменений тест priority-donate-one стал выполняться корректно. Теперь главный поток, удерживающий замок, получает временное повышение приоритета от потоков, ожидающих этот замок. В выводе видно, что фактический приоритет главного потока совпадает с ожидаемым, а потоки acquire2 и acquire1 получают доступ к замку строго в порядке убывания приоритета. После освобождения замка приоритет главного потока возвращается к исходному значению. Это подтверждает, что механизм наследования приоритета работает правильно: при блокировке более приоритетного потока его приоритет передаётся владельцу ресурса, предотвращая инверсию приоритетов и обеспечивая корректное планирование.

2) Тест priority-donate-lower

Тест priority-donate-lower проверяет корректность восстановления приоритета потока после завершения действия donation. Главный поток захватывает замок и создаёт поток с более высоким приоритетом, который блокируется при попытке получить этот же замок. В результате высокий приоритет временно передаётся главному потоку. Затем главный поток вручную понижает свой базовый приоритет, однако фактический приоритет при этом не должен измениться — он должен оставаться повышенным до тех

пор, пока удерживается замок и действует donation. После освобождения замка приоритет главного потока должен восстановиться до нового, пониженного значения. Таким образом, тест подтверждает, что механизм наследования приоритета работает корректно: donation сохраняется, пока замок не освобождён, и снимается сразу после его освобождения.

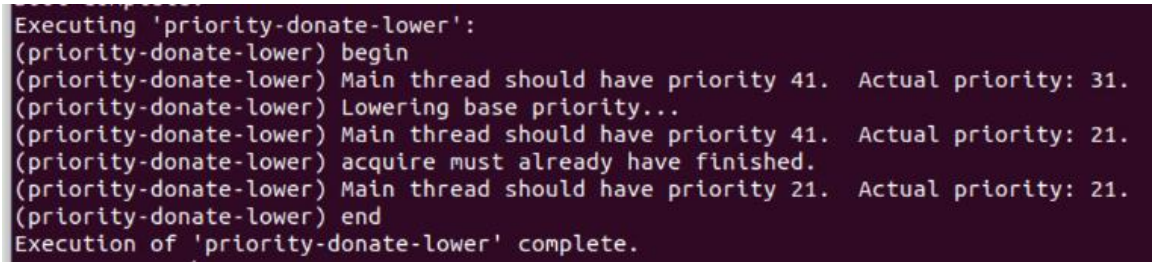


Рис. 13 «Тест priority-donate-lower до внесённых изменений»

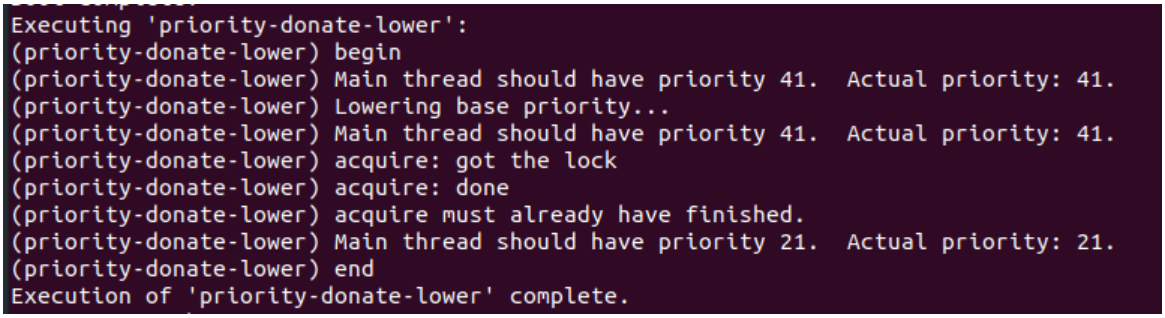


Рис. 14 «Тест priority-donate-lower после внесённых изменений»

После внесённых изменений тест **priority-donate-lower** выполняется корректно: при попытке понизить приоритет поток удерживает полученную "донатную" приоритетную оценку до тех пор, пока не освободит замок. После освобождения замка приоритет возвращается к исходному значению.

3) Тест priority-donate-multiple

Тест priority-donate-multiple проверяет работу механизма множественного наследования приоритета — ситуацию, когда один поток одновременно получает donation от нескольких других. Главный поток захватывает два замка A и B, после чего создаются два потока с более высокими приоритетами. Один поток пытается захватить замок A, а другой — замок B. Поскольку оба замка заняты главным потоком, оба ожидающих потока передают ему свои приоритеты. В результате приоритет главного потока временно повышается до уровня самого приоритетного из ожидающих. Далее главный поток последовательно освобождает замки.

После освобождения первого замка приоритет остаётся повышенным, так как второй поток всё ещё ждёт, а после освобождения второго замка приоритет главного возвращается к исходному значению. Таким образом, тест проверяет, что поток может получать приоритет от нескольких источников одновременно и корректно восстанавливать свой собственный приоритет после снятия всех donation.

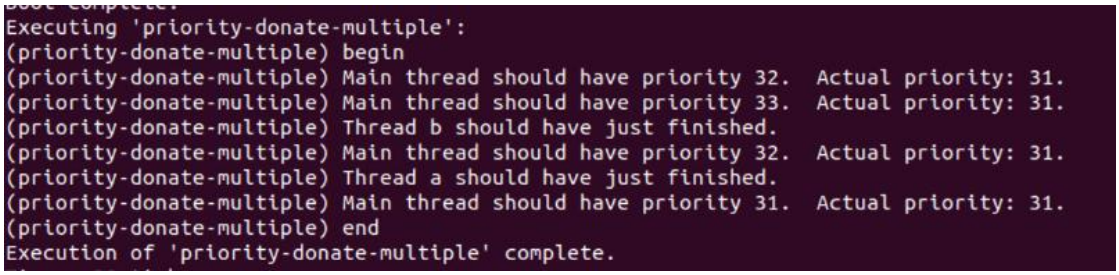


Рис. 15 «Тест priority-donate-multiple до внесённых изменений»

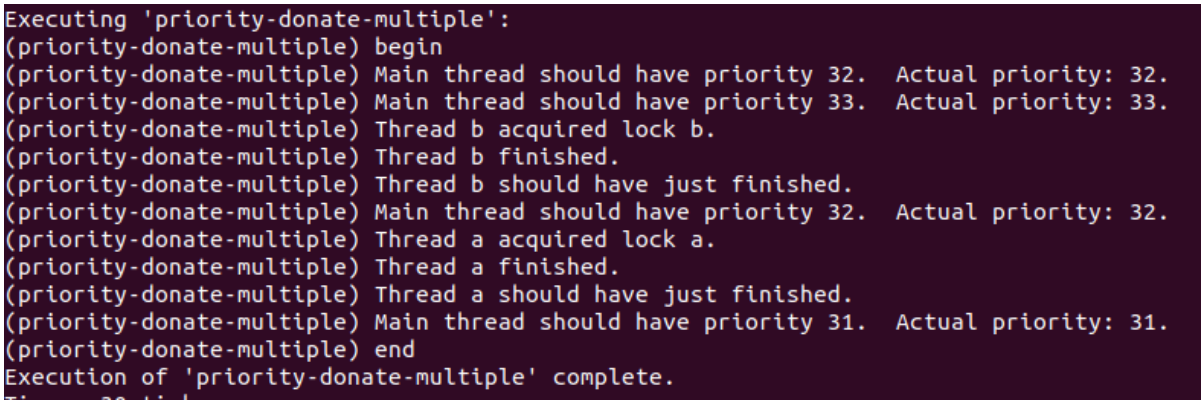


Рис. 16 «Тест priority-donate-multiple после внесённых изменений»

После внесённых изменений тест **priority-donate-multiple** проходит успешно: главный поток корректно получает несколько приоритетных "донатов" от разных потоков и восстанавливает свой исходный приоритет после освобождения всех замков. Потоки выполняются в порядке убывания приоритета, что подтверждает правильную реализацию множественного разделения приоритета.

4) Тест priority-donate-multiple2

Тест priority-donate-multiple2 проверяет, как работает механизм множественного наследования приоритета, когда один поток получает приоритет сразу от нескольких других потоков. В начале главный поток захватывает два замка — A и B. Затем создаются два потока с более высоким

приоритетом: первый (a_thread) пытается получить замок A, а второй (b_thread) — замок B. Оба потока блокируются, потому что эти замки уже удерживает главный поток. В этот момент происходит donation — оба ожидающих потока “жертвуют” свои приоритеты владельцу замков, и главный поток временно получает приоритет, равный наибольшему из переданных. После этого главный поток последовательно освобождает замки. Когда освобождается первый замок, donation от соответствующего потока снимается, но приоритет остаётся повышенным за счёт второго потока. После освобождения второго замка все donation прекращаются, и приоритет главного потока возвращается к исходному значению. Таким образом, тест подтверждает, что система корректно обрабатывает несколько одновременных donation и правильно восстанавливает приоритет после их снятия.

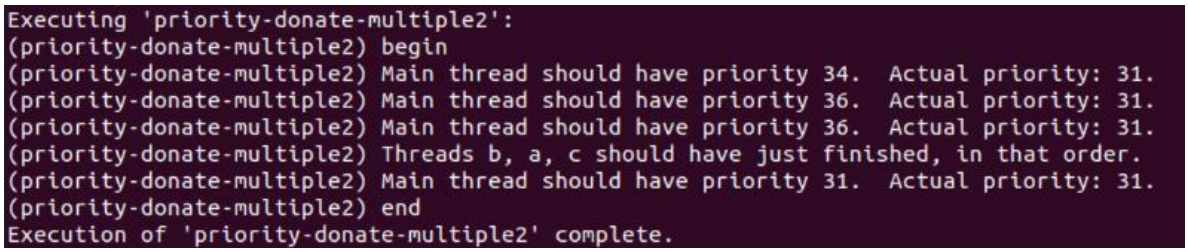


Рис. 16 «Тест priority-donate-multiple2 до внесённых изменений»

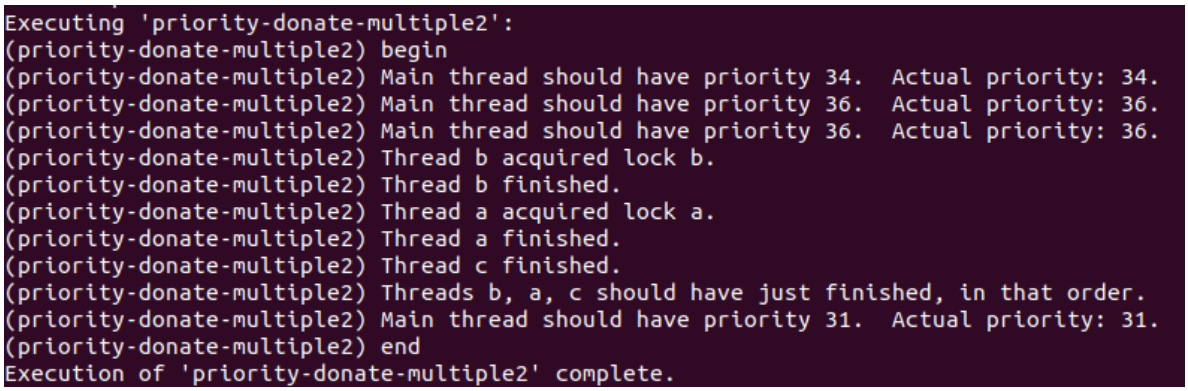


Рис. 17 «Тест priority-donate-multiple после внесённых изменений»

После внесённых изменений тест **priority-donate-multiple2** выполняется правильно: главный поток получает приоритетную "донатацию" от всех зависимых потоков, удерживающих разные замки. Потоки завершают работу в корректной последовательности — от наиболее приоритетных к

наименее приоритетным, а после освобождения всех замков главный поток возвращается к своему исходному приоритету.

5) Тест priority-donate-sema

Тест priority-donate-sema проверяет корректность работы механизма наследования приоритета при использовании семафоров и замков вместе.

В начале поток с низким приоритетом L захватывает замок и затем блокируется на sema_down(). После этого создаётся поток M со средним приоритетом, который тоже блокируется, ожидая тот же семафор. Затем запускается поток H с высоким приоритетом, который пытается захватить тот же замок, но не может, так как он уже удерживается потоком L. В этот момент поток H передаёт (donate) свой приоритет потоку L, чтобы тот мог быстрее завершить свою работу и освободить замок.

Когда главный поток вызывает sema_up(), поток L пробуждается и выполняется с уже повышенным приоритетом, освобождая замок. Это позволяет потоку H получить замок и завершить работу, после чего выполняются потоки M, L и главный.

Таким образом, тест проверяет, что donation приоритета корректно работает даже в случаях, когда потоки блокируются не напрямую на замке, а через семафор, и что приоритет корректно передаётся владельцу замка для предотвращения инверсии приоритетов.

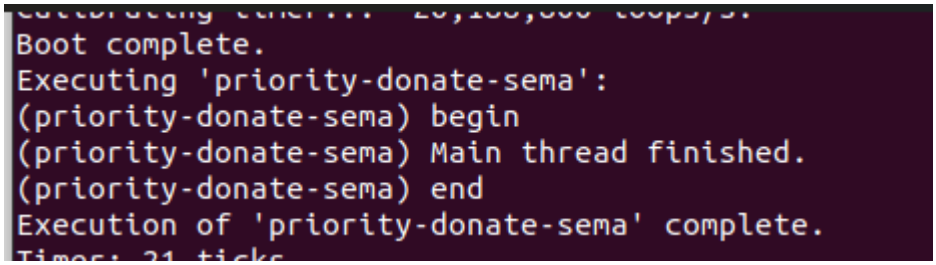


Рис. 18 «Тест priority-donate-sema до внесённых изменений»

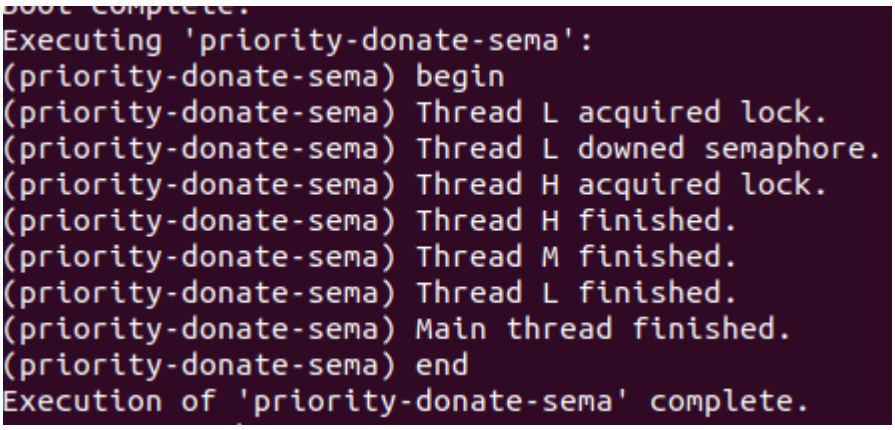


Рис. 19 «Тест priority-donate-sema после внесённых изменений»

После изменений тест **priority-donate-sema** проходит корректно: при блокировке потока H на замке, удерживаемом L, приоритет H временно передаётся L. После sema_up() L выполняется с повышенным приоритетом, быстро освобождает замок, затем завершается H, далее M, после чего — L и главный поток. Порядок подтверждает, что donation сохраняется вместе с работой семафора и снимается после освобождения замка.

6) Тест priority-donate-nest

Тест priority-donate-nest проверяет вложенное наследование приоритета — когда высокий приоритет должен “прокатиться” по цепочке из нескольких замков и потоков. Низкоприоритетный главный поток берёт замок A. Поток M (средний приоритет) берёт замок B и дальше блокируется на A. Затем поток H (высокий приоритет) пытается взять B и блокируется на M. В итоге получается цепочка зависимостей: H → M → L(main). Ожидаемое поведение: при попытке H получить B его приоритет донатится к M, а при попытке M получить A — уже повышенный приоритет переносится дальше к главному потоку. Главный поток временно получает наивысший приоритет, быстро освобождает A, после чего M завершает свои критические секции с приоритетом H, освобождает B, и в конце выполняется H. Тест подтверждает, что donation работает транзитивно по цепочке и правильно снимается при поочерёдном освобождении замков.

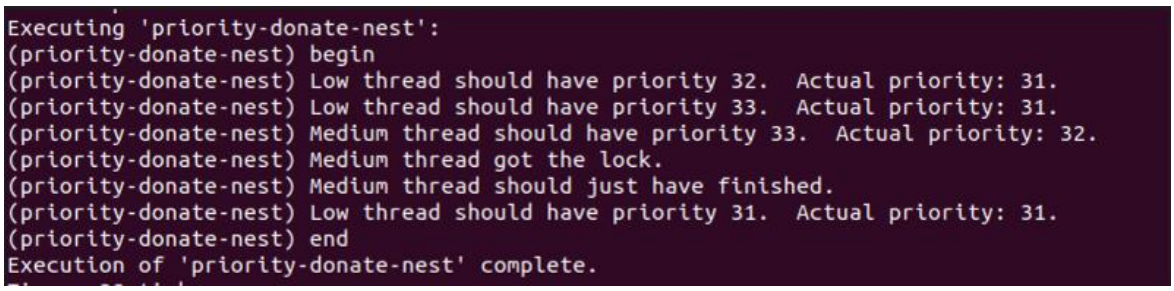


Рис. 20 «Тест priority-donate-nest до внесённых изменений»

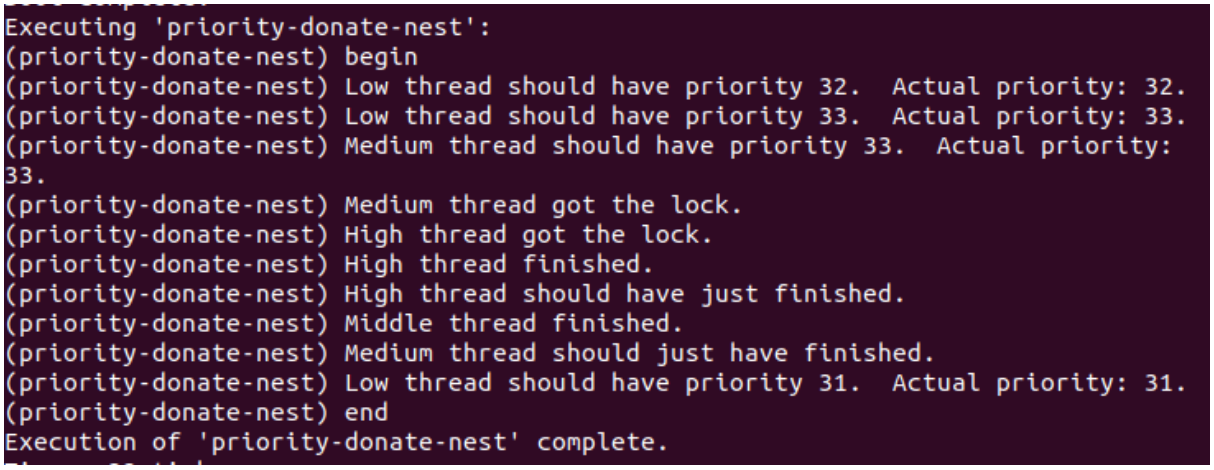


Рис. 21 «Тест priority-donate-nest после внесённых изменений»

После внесённых изменений тест **priority-donate-nest** проходит успешно: при вложенном ожидании замков приоритет корректно передаётся по цепочке — от потока H к M, затем к L. Потоки завершаются в правильной последовательности — сначала H, потом M, и последним L. Это подтверждает корректную работу механизма вложенного (цепного) разделения приоритета.

7) **Тест priority-donate-chain**

Тест priority-donate-chain проверяет, что передача приоритета правильно «прокатывается» через длинную цепочку ожиданий замков и что посторонние потоки с чуть меньшим приоритетом не прерывают эту цепочку. Главный поток ставит себе PRI_MIN, инициализирует набор замков locks[0..N-1] и захватывает lock0.

Далее создаются донор-потоки thread1..threadN с возрастающими приоритетами (шаг +3). Для каждого задают пару замков (first, second): он сначала берёт first (свой замок), а потом пытается взять second — замок предыдущего в цепочке (или lock0 у главного). Из-за second поток

блокируется и донатит свой приоритет владельцу second. Так формируется цепь зависимостей: threadN → … → thread1 → main, и приоритет с вершины должен дойти до главного потока.

После создания всей цепочки главный поток освобождает lock0. Управление начинает “сползать” вниз: thread1 просыпается, берёт/отпускает свои замки и разблокирует thread2, затем thread2 — thread3, и так до threadN. При этом на каждом шаге donation у уже освободившегося участника снимается, а у ещё блокирующих сохраняется.

Параллельно создаются interloper-потоки с приоритетом на 1 ниже, чем у соответствующих доноров. Они должны завершаться только после того, как отработает их “старший” донор — это подтверждает, что цепной donation действительно поднимает нужных участников выше любых конкурентов. Итог: приоритет транзитивно передаётся по цепочке, корректно снимается при release, и более слабые конкуренты не вмешиваются.



Рис. 22 «Тест priority-donate-chain до внесённых изменений»

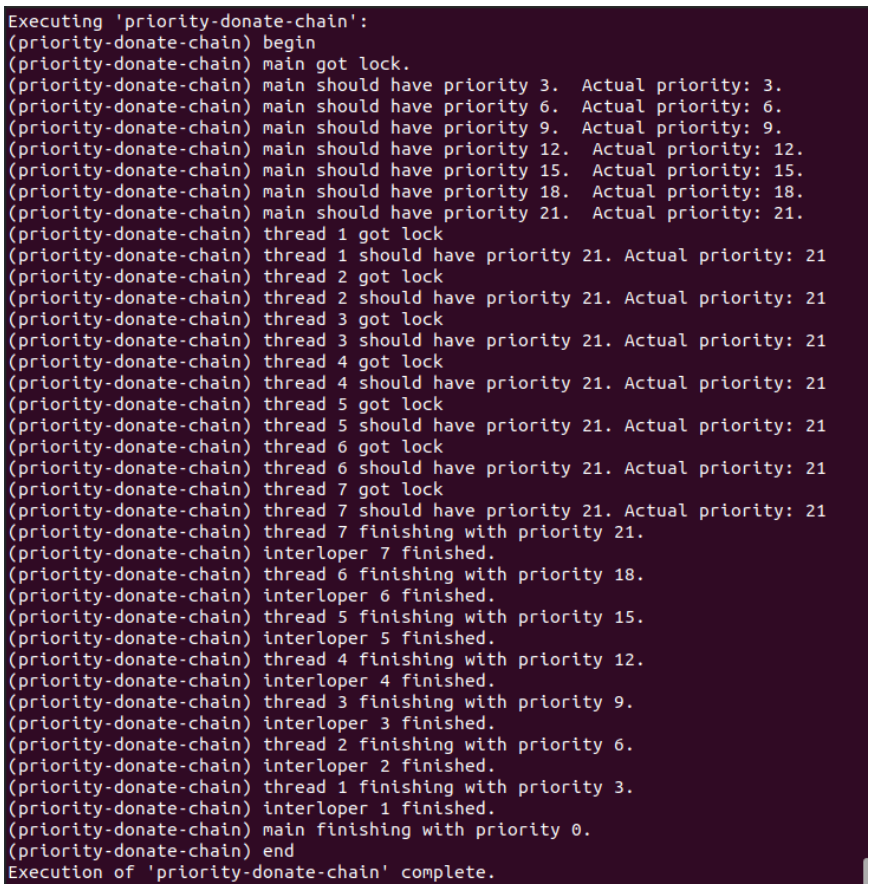


Рис. 22 «Тест priority-donate-chain после внесённых изменений»

После внесённых изменений тест **priority-donate-chain** выполняется успешно: при последовательной блокировке замков приоритет правильно передаётся по цепочке от последнего потока к первому, включая основной поток. Все уровни приоритета обновляются корректно на каждом этапе цепочки, а после освобождения замков приоритеты возвращаются к исходным значениям. Это подтверждает корректную работу многоуровневого (цепного) механизма разделения приоритетов.

Была составлена диаграмма исполнения процессов для нового алгоритма планирования.

Процесс:	CPU burst:	Приоритет процесса:
Proc0	2	45
Proc1	2	44
Proc2	2	46
Proc3	3	33
Proc4	10	33

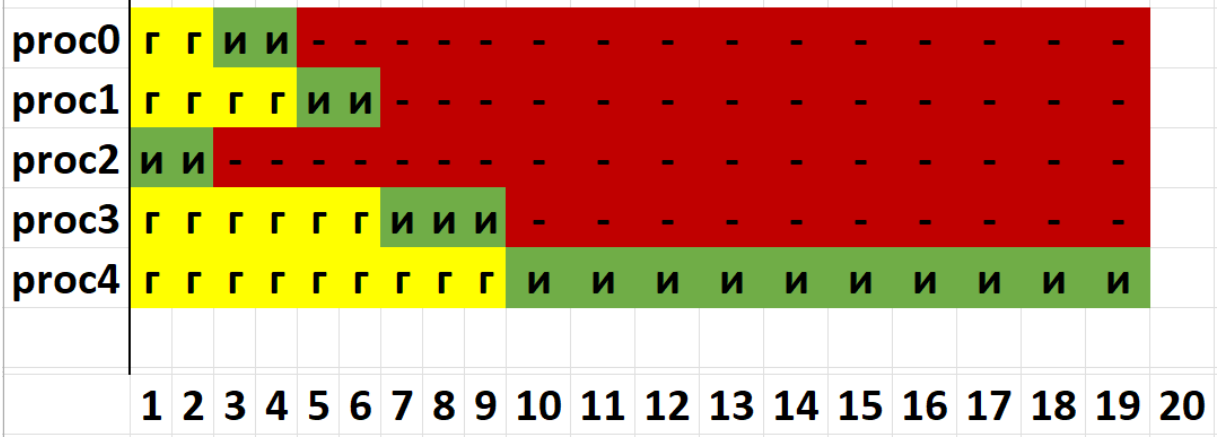


Рис. 23 «Диаграмма исполнения для нового алгоритма планирования»

Среднее время ожидания:

Tср. вр ожид = Σ(г) / N = 21 / 5 = 4,2 кванта

Полное среднее время исполнения:

Tполн ср исполн = Σ(г + и) / N = 40 / 5 = 8 квантов

Среднее время ожидания, среднее время полного исполнения не изменилось, но процессы с более высоким приоритетом выполняются раньше тех, у которых приоритет ниже.

Была составлена блок-схема нового алгоритма приоритетного планирования.

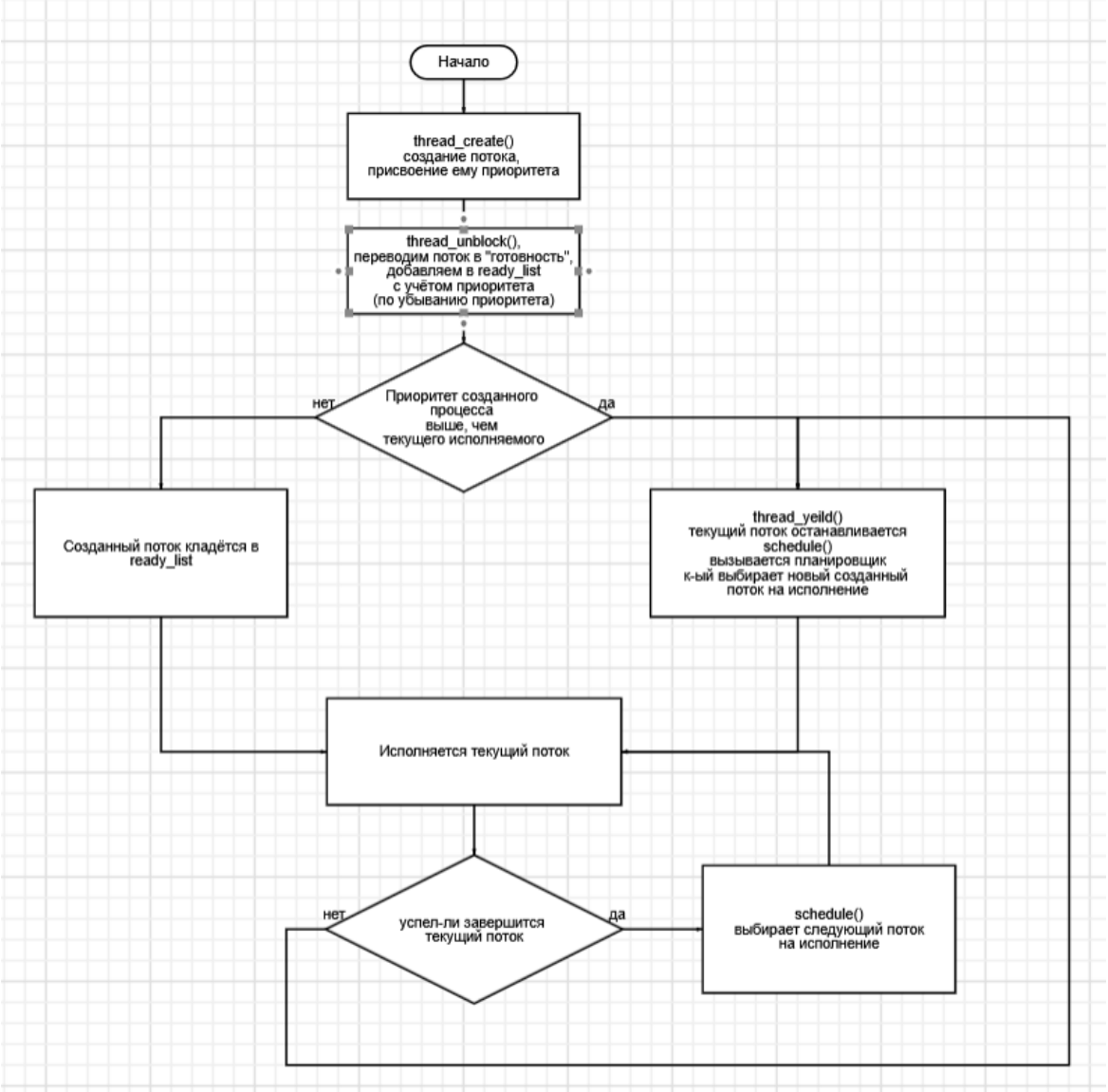


Рис. 24 «Блок схема нового алгоритма приоритетного планирования»

Полное описание всех функций планировщика было оформлено в виде таблицы.

Функция	Аргументы и их описание	Назначение и изменения
thread_create()	1) name — имя создаваемого потока. 2) priority — приоритет нового потока. 3) function — указатель на функцию, выполняемую потоком. 4) aux — дополнительный аргумент, передаваемый в функцию потока.	Создаёт новый поток. Если приоритет нового потока выше текущего — выполняется thread_yield() для переключения на него.
thread_unblock()	1) t — поток, который нужно разблокировать.	Добавляет поток в ready_list с учётом приоритета. Если разблокированный поток имеет больший приоритет, вызывается yield().
thread_yield()	—	Текущий поток

		освобождает CPU и помещается обратно в ready_list по приоритету.
thread_set_priority()	1) new_priority — новый приоритет текущего потока.	Обновляет приоритет потока, учитывая donation. Если в ready_list есть поток с большим приоритетом, вызывается yield().
thread_priority_donate()	1) donor — поток, который передаёт приоритет. 2) donated_priority — значение передаваемого приоритета.	Выполняет передачу приоритета от потока-донора к потоку, владеющему замком.
comparator_greater_thread_priority()	1) a, b — элементы списка потоков. 2) aux — вспомогательный аргумент (не используется).	Функция сравнения приоритетов потоков, применяется при сортировке списков ожидания.
sema_down()	1) sema — указатель на семафор.	Если значение семафора равно нулю, поток блокируется и добавляется в

		список ожидания с сортировкой по приоритету.
sema_up()	1) sema — указатель на семафор.	Увеличивает значение семафора и разблокирует поток с наибольшим приоритетом из списка ожидания.
lock_acquire()	1) lock — указатель на замок.	Попытка захвата замка. Если замок занят, приоритет потока передаётся владельцу замка (donation).
lock_release()	1) lock — указатель на замок.	Освобождает замок, снимает donation и восстанавливает исходный приоритет потока.
cond_wait()	1) cond — условная переменная. 2) lock — замок, который освобождается во время ожидания.	Блокирует поток и помещает его в очередь ожидания condition по приоритету.
cond_signal()	1) cond —	Пробуждает поток

	условная переменная. 2) lock — замок, связанный с условием.	с наибольшим приоритетом из очереди ожидания condition.
struct thread	—	Добавлены поля: 1) original_priority — исходный приоритет потока. 2) waiting_lock — замок, который поток ожидает. 3) locks — список захваченных замков.
struct semaphore / struct lock	—	Добавлены поля для хранения приоритета и связи между потоками при блокировке (для реализации donation). **value** — хранит текущее значение семафора (счётчик доступных ресурсов). **waiters** — список потоков, ожидающих

		освобождения ресурса. **priority** — максимальный приоритет среди ожидающих потоков; нужен, чтобы при sema_up() пробуждался поток с наивысшим приоритетом.

ВЫВОД

В ходе выполнения лабораторной работы был реализован приоритетный планировщик потоков в операционной системе PintOS с поддержкой механизма разделения приоритетов (priority donation). После внесённых изменений все тесты на приоритетное планирование и наследование приоритета (alarm-priority, priority-change, priority-preempt, priority-sema, priority-condvar, priority-donate-one и др.) успешно прошли. Реализованный алгоритм обеспечивает корректное выполнение потоков в порядке приоритетов, предотвращает инверсию приоритетов и сохраняет справедливое чередование (round-robin) между потоками с одинаковыми приоритетами. Средние показатели времени ожидания и полного времени выполнения остались прежними, однако система стала более предсказуемой и устойчивой к блокировкам низкоприоритетных потоков.

Листинг А «thread.h»

```
#ifndef THREADS_THREAD_H
#define THREADS_THREAD_H

#include <debug.h>
#include <list.h>
#include <stdint.h>

/* States in a thread's life cycle. */
enum thread_status
  {
    THREAD_RUNNING,     /* Поток сейчас выполняется на ЦПУ  */
    THREAD_READY,         /* Не выполняется, но готов (ждёт в очереди
ready_list) */
    THREAD_BLOCKED,                 /*   Заблокирован,   ждёт   событие
(семафор/замок/cond/timer) */
    THREAD_DYING        /* Завершён, ждёт уничтожения структур */
  };

/* Thread identifier type.
   You can redefine this to whatever type you like. */
typedef int tid_t;
#define TID_ERROR ((tid_t) -1)          /* Error value for tid_t. */

/* Thread priorities. */
#define PRI_MIN 0                       /* Lowest priority. */
#define PRI_DEFAULT 31                  /* Default priority. */
#define PRI_MAX 63                      /* Highest priority. */

/* A kernel thread or user process.

   Each thread structure is stored in its own 4 kB page.  The
   thread structure itself sits at the very bottom of the page
   (at offset 0).  The rest of the page is reserved for the
   thread's kernel stack, which grows downward from the top of
   the page (at offset 4 kB).  Here's an illustration:

        4 kB +---------------------------------+
             |          kernel stack           |
             |                |                |
             |                |                |
             |                V                |
             |         grows downward          |
             |                                 |
             |                                 |
             |                                 |
             |                                 |
             |                                 |
             |                                 |
             |                                 |
             |                                 |
             +---------------------------------+
             |              magic              |
             |                :                |
             |                :                |
             |               name              |
             |              status             |
```

```
        0 kB +---------------------------------+
```

The upshot of this is twofold:

1. First, `struct thread' must not be allowed to grow too big. If it does, then there will not be enough room for the kernel stack. Our base `struct thread' is only a few bytes in size. It probably should stay well under 1 kB.

2. Second, kernel stacks must not be allowed to grow too large. If a stack overflows, it will corrupt the thread state. Thus, kernel functions should not allocate large structures or arrays as non-static local variables. Use dynamic allocation with malloc() or palloc_get_page() instead.

The first symptom of either of these problems will probably be an assertion failure in thread_current(), which checks that the `magic' member of the running thread's `struct thread' is set to THREAD_MAGIC. Stack overflow will normally change this value, triggering the assertion. */

```
/* The `elem' member has a dual purpose.  It can be an element in
   the run queue (thread.c), or it can be an element in a
   semaphore wait list (synch.c).  It can be used these two ways
   only because they are mutually exclusive: only a thread in the
   ready state is on the run queue, whereas only a thread in the
   blocked state is on a semaphore wait list. */
struct thread
  {
    /* Owned by thread.c. */
    tid_t tid;                          /* Thread identifier. */
    enum thread_status status;          /* Thread state. */
    char name[16];                              /* Name (for debugging
purposes). */
    uint8_t *stack;                     /* Saved stack pointer. */
    int priority;                       /* Priority. */
    int original_priority;                 /* priority, before donation
*/

    struct list_elem allelem;              /* Узел двусвязного списка,
для глоб-ого сп-ка прц-ов */


    /* Shared between thread.c and synch.c. */
    struct list_elem elem;                  /* Узел двусв. списка для
динам. очередей(ready лист) */

    // need for priority donations
    struct lock* waiting_lock;  // объект блокировки,к-ый ожидает этот
поток
    struct list locks; //list of locks the thread holds

#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir;                  /* Page directory. */
#endif
```

```
    /* Owned by thread.c. */
    unsigned magic;                     /* Detects stack overflow. */
  };

/* If false (default), use round-robin scheduler.
   If true, use multi-level feedback queue scheduler.
   Controlled by kernel command-line option "-o mlfqs". */
extern bool thread_mlfqs;

void thread_init (void);
void thread_start (void);

void thread_tick (void);
void thread_print_stats (void);

typedef void thread_func (void *aux);
tid_t thread_create (const char *name, int priority, thread_func *,
void *);

void thread_block (void);
void thread_unblock (struct thread *);

struct thread *thread_current (void);
tid_t thread_tid (void);
const char *thread_name (void);

void thread_exit (void) NO_RETURN;
void thread_yield (void);

/* Performs some operation on thread t, given auxiliary data AUX. */
typedef void thread_action_func (struct thread *t, void *aux);
void thread_foreach (thread_action_func *, void *);

int thread_get_priority (void);
void thread_set_priority (int);
bool priority_less(const struct list_elem* e1, const struct list_elem*
e2, void* aux);
void priority_ordering();
int priority_sema(struct list_elem* x, struct list_elem* y, void*
aux);
void thread_priority_donate(struct thread* target, int new_priority);

int thread_get_nice (void);
void thread_set_nice (int);
int thread_get_recent_cpu (void);
int thread_get_load_avg (void);

#endif /* threads/thread.h */
```

Листинг Б «thread.c»

```
#include "threads/thread.h"
#include <debug.h>
#include <stddef.h>
#include <random.h>
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
#include "threads/flags.h"
#include "threads/interrupt.h"
#include "threads/intr-stubs.h"
#include "threads/palloc.h"
#include "threads/switch.h"
#include "threads/synch.h"
#include "threads/vaddr.h"
#include "list.h"
#ifdef USERPROG
#include "userprog/process.h"
#endif

/* Random value for struct thread's `magic' member.
   Used to detect stack overflow.  See the big comment at the top
   of thread.h for details. */
#define THREAD_MAGIC 0xcd6abf4b

/* List of processes in THREAD_READY state, that is, processes
   that are ready to run but not actually running. */
static struct list ready_list;


static struct list wait_list;

/* List of all processes.  Processes are added to this list
   when they are first scheduled and removed when they exit. */
static struct list all_list;

/* Idle thread. */
static struct thread *idle_thread;

/* Initial thread, the thread running init.c:main(). */
static struct thread *initial_thread;

/* Lock used by allocate_tid(). */
static struct lock tid_lock;

/* Stack frame for kernel_thread(). */
struct kernel_thread_frame
  {
    void *eip;                  /* Return address. */
    thread_func *function;      /* Function to call. */
    void *aux;                  /* Auxiliary data for function. */
  };

/* Statistics. */
static long long idle_ticks;    /* # of timer ticks spent idle. */
static long long kernel_ticks;  /* # of timer ticks in kernel threads.
*/
```

```
static long long user_ticks;    /* # of timer ticks in user programs.
*/

/* Scheduling. */
#define TIME_SLICE 4                /* # of timer ticks to give each
thread. */
static unsigned thread_ticks;   /* # of timer ticks since last yield.
*/

/* If false (default), use round-robin scheduler.
   If true, use multi-level feedback queue scheduler.
   Controlled by kernel command-line option "-o mlfqs". */
bool thread_mlfqs;

static void kernel_thread (thread_func *, void *aux);

static void idle (void *aux UNUSED);
static struct thread *running_thread (void);
static struct thread *next_thread_to_run (void);
static void init_thread (struct thread *, const char *name, int
priority);
static bool is_thread (struct thread *) UNUSED;
static void *alloc_frame (struct thread *, size_t size);
static void schedule (void);
void thread_schedule_tail (struct thread *prev);
static tid_t allocate_tid (void);
static bool
comparator_greater_thread_priority(
  const struct list_elem* a,
  const struct list_elem* b, void* aux UNUSED);

/* Initializes the threading system by transforming the code
   that's currently running into a thread.  This can't work in
   general and it is possible in this case only because loader.S
   was careful to put the bottom of the stack at a page boundary.

   Also initializes the run queue and the tid lock.

   After calling this function, be sure to initialize the page
   allocator before trying to create any threads with
   thread_create().

   It is not safe to call thread_current() until this function
   finishes. */
void
thread_init (void)
{
  ASSERT (intr_get_level () == INTR_OFF);

  lock_init (&tid_lock);
  list_init (&ready_list);
  list_init (&wait_list);
  list_init (&all_list);

  /* Set up a thread structure for the running thread. */
  initial_thread = running_thread ();
  init_thread (initial_thread, "main", PRI_DEFAULT);
  initial_thread->status = THREAD_RUNNING;
```

```
  initial_thread->tid = allocate_tid ();
}

/* Starts preemptive thread scheduling by enabling interrupts.
   Also creates the idle thread. */
void
thread_start (void)
{
  /* Create the idle thread. */
  struct semaphore idle_started;
  sema_init (&idle_started, 0);
  thread_create ("idle", PRI_MIN, idle, &idle_started);

  /* Start preemptive thread scheduling. */
  intr_enable ();

  /* Wait for the idle thread to initialize idle_thread. */
  sema_down (&idle_started);
}

/* Called by the timer interrupt handler at each timer tick.
   Thus, this function runs in an external interrupt context. */
void
thread_tick (void)
{
  struct thread *t = thread_current ();

  /* Update statistics. */
  if (t == idle_thread)
    idle_ticks++;
#ifdef USERPROG
  else if (t->pagedir != NULL)
    user_ticks++;
#endif
  else
    kernel_ticks++;

  /* Enforce preemption. */
  if (++thread_ticks >= TIME_SLICE)
    intr_yield_on_return ();
}

/* Prints thread statistics. */
void
thread_print_stats (void)
{
  printf ("Thread: %lld idle ticks, %lld kernel ticks, %lld user
ticks\n",
          idle_ticks, kernel_ticks, user_ticks);
}

/* Creates a new kernel thread named NAME with the given initial
   PRIORITY, which executes FUNCTION passing AUX as the argument,
   and adds it to the ready queue.  Returns the thread identifier
   for the new thread, or TID_ERROR if creation fails.

   If thread_start() has been called, then the new thread may be
   scheduled before thread_create() returns.  It could even exit
```

```
   before thread_create() returns.  Contrariwise, the original
   thread may run for any amount of time before the new thread is
   scheduled.  Use a semaphore or some other form of
   synchronization if you need to ensure ordering.

   The code provided sets the new thread's `priority' member to
   PRIORITY, but no actual priority scheduling is implemented.
   Priority scheduling is the goal of Problem 1-3. */
tid_t
thread_create (const char *name, int priority,
               thread_func *function, void *aux)
{
  struct thread *t;
  struct kernel_thread_frame *kf;
  struct switch_entry_frame *ef;
  struct switch_threads_frame *sf;
  tid_t tid;


  ASSERT (function != NULL);

  /* Allocate thread. */
  t = palloc_get_page (PAL_ZERO);
  if (t == NULL)
    return TID_ERROR;

  /* Initialize thread. */
  init_thread (t, name, priority);
  tid = t->tid = allocate_tid ();


  /* Stack frame for kernel_thread(). */
  kf = alloc_frame (t, sizeof *kf);
  kf->eip = NULL;
  kf->function = function;
  kf->aux = aux;

  /* Stack frame for switch_entry(). */
  ef = alloc_frame (t, sizeof *ef);
  ef->eip = (void (*) (void)) kernel_thread;

  /* Stack frame for switch_threads(). */
  sf = alloc_frame (t, sizeof *sf);
  sf->eip = switch_entry;
  sf->ebp = 0;



  /* Add to run queue. */
  thread_unblock (t);

  if (priority > thread_current()->priority)
  {
    thread_yield();
  }

  return tid;
}
```

```
/* Puts the current thread to sleep.  It will not be scheduled
   again until awoken by thread_unblock().

   This function must be called with interrupts turned off.  It
   is usually a better idea to use one of the synchronization
   primitives in synch.h. */
void
thread_block (void)
{
  ASSERT (!intr_context ());
  ASSERT (intr_get_level () == INTR_OFF);

  thread_current ()->status = THREAD_BLOCKED;
  schedule ();
}

/* Transitions a blocked thread T to the ready-to-run state.
   This is an error if T is not blocked.  (Use thread_yield() to
   make the running thread ready.)

   This function does not preempt the running thread.  This can
   be important: if the caller had disabled interrupts itself,
   it may expect that it can atomically unblock a thread and
   update other data. */
```

```
void priority_ordering()
{
  struct thread* t =thread_current();
  enum intr_level intt= intr_disable();
  if (!list_empty(&ready_list))
  {
    struct  thread  *y  =  list_entry(list_front(&ready_list),  struct
thread, elem);

    if (t->priority < y->priority)
    {
      thread_yield();
    }
  }
  intr_set_level(intt);
}
```

```
void
thread_unblock (struct thread *t)
{
  enum intr_level old_level;

  ASSERT (is_thread (t));

  old_level = intr_disable ();
  ASSERT (t->status == THREAD_BLOCKED);
```

```
  list_insert_ordered(&ready_list,                          &t->elem,
comparator_greater_thread_priority, NULL); /*******************/
  t->status = THREAD_READY;

  if (thread_current() != idle_thread && thread_current()->priority <
t->priority)
  {
    thread_yield();
  }
  intr_set_level (old_level);
}

/* Returns the name of the running thread. */
const char *
thread_name (void)
{
  return thread_current ()->name;
}

/* Returns the running thread.
   This is running_thread() plus a couple of sanity checks.
   See the big comment at the top of thread.h for details. */
struct thread *
thread_current (void)
{
  struct thread *t = running_thread ();

  /* Make sure T is really a thread.
     If either of these assertions fire, then your thread may
     have overflowed its stack.  Each thread has less than 4 kB
     of stack, so a few big automatic arrays or moderate
     recursion can cause stack overflow. */
  ASSERT (is_thread (t));
  ASSERT (t->status == THREAD_RUNNING);

  return t;
}

/* Returns the running thread's tid. */
tid_t
thread_tid (void)
{
  return thread_current ()->tid;
}

/* Deschedules the current thread and destroys it.  Never
   returns to the caller. */
void
thread_exit (void)
{
  ASSERT (!intr_context ());

#ifdef USERPROG
  process_exit ();
#endif

  /* Remove thread from all threads list, set our status to dying,
     and schedule another process.  That process will destroy us
```

```
     when it calls thread_schedule_tail(). */
  intr_disable ();
  list_remove (&thread_current()->allelem);
  thread_current ()->status = THREAD_DYING;
  schedule ();
  NOT_REACHED ();
}

/* Yields the CPU.  The current thread is not put to sleep and
   may be scheduled again immediately at the scheduler's whim. */
void
thread_yield (void)
{
  struct thread *cur = thread_current ();
  enum intr_level old_level;

  ASSERT (!intr_context ());

  old_level = intr_disable ();
  if (cur != idle_thread){
    list_insert_ordered(&ready_list,&cur->elem,
comparator_greater_thread_priority, NULL);
  }
  cur->status = THREAD_READY;
  schedule();
  intr_set_level (old_level);
}

/* Invoke function 'func' on all threads, passing along 'aux'.
   This function must be called with interrupts off. */
void
thread_foreach (thread_action_func *func, void *aux)
{
  struct list_elem *e;

  ASSERT (intr_get_level () == INTR_OFF);

  for (e = list_begin (&all_list); e != list_end (&all_list);
       e = list_next (e))
    {
      struct thread *t = list_entry (e, struct thread, allelem);
      func (t, aux);
    }
}

/* Sets the current thread's priority to NEW_PRIORITY. */
void
thread_set_priority   (int   new_priority)                              //
************************
{
  struct thread* t_current = thread_current();
  if( t_current->priority == t_current->original_priority)
  {
    t_current->priority = new_priority;
    t_current->original_priority = new_priority;
  }
  else
  {
```

```
    t_current->original_priority=new_priority;
  }


  if (!list_empty(&ready_list))
  {
    struct    thread*    next=list_entry(list_begin(&ready_list),struct
thread,elem);
    if(next !=NULL && next->priority > new_priority)
    {
      thread_yield();
    }
  }
}

void thread_priority_donate(struct thread* target, int new_priority)
{
  // change only current priority
  target->priority = new_priority;

  //if current thread gets its priority decreased, then yield
  if(target == thread_current() && !list_empty(&ready_list))
  {
    struct  thread*  next  =  list_entry(list_begin(&ready_list),struct
thread,elem);
    if (next !=NULL && next->priority > new_priority)
    {
      thread_yield();
    }
  }
}

/* Returns the current thread's priority. */
int
thread_get_priority (void)
{
  return thread_current ()->priority;
}

/* Sets the current thread's nice value to NICE. */
void
thread_set_nice (int nice UNUSED)
{
  /* Not yet implemented. */
}

/* Returns the current thread's nice value. */
int
thread_get_nice (void)
{
  /* Not yet implemented. */
  return 0;
}

/* Returns 100 times the system load average. */
int
thread_get_load_avg (void)
{
```

```
  /* Not yet implemented. */
  return 0;
}

/* Returns 100 times the current thread's recent_cpu value. */
int
thread_get_recent_cpu (void)
{
  /* Not yet implemented. */
  return 0;
}
/* Idle thread.  Executes when no other thread is ready to run.

   The idle thread is initially put on the ready list by
   thread_start().  It will be scheduled once initially, at which
   point it initializes idle_thread, "up"s the semaphore passed
   to it to enable thread_start() to continue, and immediately
   blocks.  After that, the idle thread never appears in the
   ready list.  It is returned by next_thread_to_run() as a
   special case when the ready list is empty. */
static void
idle (void *idle_started_ UNUSED)
{
  struct semaphore *idle_started = idle_started_;
  idle_thread = thread_current ();
  sema_up (idle_started);

  for (;;)
    {
      /* Let someone else run. */
      intr_disable ();
      thread_block ();

      /* Re-enable interrupts and wait for the next one.

         The `sti' instruction disables interrupts until the
         completion of the next instruction, so these two
         instructions are executed atomically.  This atomicity is
         important; otherwise, an interrupt could be handled
         between re-enabling interrupts and waiting for the next
         one to occur, wasting as much as one clock tick worth of
         time.

         See [IA32-v2a] "HLT", [IA32-v2b] "STI", and [IA32-v3a]
         7.11.1 "HLT Instruction". */
      asm volatile ("sti; hlt" : : : "memory");
    }
}
/* Function used as the basis for a kernel thread. */
static void
kernel_thread (thread_func *function, void *aux)
{
  ASSERT (function != NULL);

  intr_enable ();       /* The scheduler runs with interrupts off. */
  function (aux);       /* Execute the thread function. */
  thread_exit ();       /* If function() returns, kill the thread. */
}
```

```
/* Returns the running thread. */
struct thread *
running_thread (void)
{
  uint32_t *esp;
  /* Copy the CPU's stack pointer into `esp', and then round that
     down to the start of a page.  Because `struct thread' is
     always at the beginning of a page and the stack pointer is
     somewhere in the middle, this locates the curent thread. */
  asm ("mov %%esp, %0" : "=g" (esp));
  return pg_round_down (esp);
}

/* Returns true if T appears to point to a valid thread. */
static bool
is_thread (struct thread *t)
{
  return t != NULL && t->magic == THREAD_MAGIC;
}
/* Does basic initialization of T as a blocked thread named
   NAME. */
static void
init_thread (struct thread *t, const char *name, int priority)
{
  enum intr_level old_level;

  ASSERT (t != NULL);
  ASSERT (PRI_MIN <= priority && priority <= PRI_MAX);
  ASSERT (name != NULL);

  memset (t, 0, sizeof *t);
  t->status = THREAD_BLOCKED;
  strlcpy (t->name, name, sizeof t->name);
  t->stack = (uint8_t *) t + PGSIZE;
  t->priority = priority;
  t->original_priority= priority;
  t->waiting_lock = NULL;
  list_init(&t->locks);
  t->magic = THREAD_MAGIC;

  old_level = intr_disable();
  list_push_back (&all_list, &t->allelem);
  intr_set_level(old_level);
}
/* Allocates a SIZE-byte frame at the top of thread T's stack and
   returns a pointer to the frame's base. */
static void *
alloc_frame (struct thread *t, size_t size)
{
  /* Stack data is always allocated in word-size units. */
  ASSERT (is_thread (t));
  ASSERT (size % sizeof (uint32_t) == 0);

  t->stack -= size;
  return t->stack;
}

/* Chooses and returns the next thread to be scheduled.  Should
```

```
   return a thread from the run queue, unless the run queue is
   empty.  (If the running thread can continue running, then it
   will be in the run queue.)  If the run queue is empty, return
   idle_thread. */
static struct thread *
next_thread_to_run (void)
{
  if (list_empty (&ready_list))
    return idle_thread;
  else
    return list_entry (list_pop_front (&ready_list), struct thread,
elem);
}
/* Completes a thread switch by activating the new thread's page
   tables, and, if the previous thread is dying, destroying it.

   At this function's invocation, we just switched from thread
   PREV, the new thread is already running, and interrupts are
   still disabled.  This function is normally invoked by
   thread_schedule() as its final action before returning, but
   the first time a thread is scheduled it is called by
   switch_entry() (see switch.S).

   It's not safe to call printf() until the thread switch is
   complete.  In practice that means that printf()s should be
   added at the end of the function.

   After this function and its caller returns, the thread switch
   is complete. */
void
thread_schedule_tail (struct thread *prev)
{
  struct thread *cur = running_thread ();

  ASSERT (intr_get_level () == INTR_OFF);

  /* Mark us as running. */
  cur->status = THREAD_RUNNING;

  /* Start new time slice. */
  thread_ticks = 0;
#ifdef USERPROG
  /* Activate the new address space. */
  process_activate ();
#endif

  /* If the thread we switched from is dying, destroy its struct
     thread.  This must happen late so that thread_exit() doesn't
     pull out the rug under itself.  (We don't free
     initial_thread because its memory was not obtained via
     palloc().) */
  if (prev != NULL && prev->status == THREAD_DYING && prev !=
initial_thread)
    {
      ASSERT (prev != cur);
      palloc_free_page (prev);
    }
}
```

```
/* Schedules a new process.  At entry, interrupts must be off and
   the running process's state must have been changed from
   running to some other state.  This function finds another
   thread to run and switches to it.

   It's not safe to call printf() until thread_schedule_tail()
   has completed. */
static void
schedule (void)
{
  struct thread *cur = running_thread ();
  struct thread *next = next_thread_to_run ();
  struct thread *prev = NULL;

  ASSERT (intr_get_level () == INTR_OFF);
  ASSERT (cur->status != THREAD_RUNNING);
  ASSERT (is_thread (next));

  if (cur != next)
    prev = switch_threads (cur, next);
  thread_schedule_tail (prev);
}
/* Returns a tid to use for a new thread. */
static tid_t
allocate_tid (void)
{
  static tid_t next_tid = 1;
  tid_t tid;

  lock_acquire (&tid_lock);
  tid = next_tid++;
  lock_release (&tid_lock);

  return tid;
}
static bool
comparator_greater_thread_priority(
  const struct list_elem* a,
  const struct list_elem* b, void* aux UNUSED)
{
  struct thread *ta, *tb;
  ASSERT(a!= NULL);
  ASSERT(b!=NULL);
  ta = list_entry(a,struct thread, elem);
  tb = list_entry(b,struct thread, elem);
  return ta->priority > tb->priority;
}


/* Offset of `stack' member within `struct thread'.
   Used by switch.S, which can't figure it out on its own. */
uint32_t thread_stack_ofs = offsetof (struct thread, stack);
```

Листинг С «synch.h»

```
#ifndef THREADS_SYNCH_H
#define THREADS_SYNCH_H

#include <list.h>
#include <stdbool.h>

/* A counting semaphore. */
struct semaphore
  {
    unsigned value;             /* Current value. */
    struct list waiters;        /* List of waiting threads. */
    int priority;
  };

void sema_init (struct semaphore *, unsigned value);
void sema_down (struct semaphore *);
bool sema_try_down (struct semaphore *);
void sema_up (struct semaphore *);
void sema_self_test (void);

/* Lock. */
struct lock
  {
    struct  thread  *holder;            /*  Thread  holding  lock  (for
debugging). */
    struct  semaphore  semaphore;  /*  Binary  semaphore  controlling
access. */


    struct list_elem lockelem;
    int priority;
  };

void lock_init (struct lock *);
void lock_acquire (struct lock *);
bool lock_try_acquire (struct lock *);
void lock_release (struct lock *);
bool lock_held_by_current_thread (const struct lock *);

/* Condition variable. */
struct condition
  {
    struct list waiters;        /* List of waiting threads. */
  };

void cond_init (struct condition *);
void cond_wait (struct condition *, struct lock *);
void cond_signal (struct condition *, struct lock *);
void cond_broadcast (struct condition *, struct lock *);
/* Optimization barrier.
   The compiler will not reorder operations across an
   optimization barrier.  See "Optimization Barriers" in the
   reference guide for more information.*/
#define barrier() asm volatile ("" : : : "memory")

#endif /* threads/synch.h */
```

Листинг Д «synch.c»

```
/* This file is derived from source code for the Nachos
   instructional operating system.  The Nachos copyright notice
   is reproduced in full below. */

/* Copyright (c) 1992-1996 The Regents of the University of
California.
   All rights reserved.

   Permission to use, copy, modify, and distribute this software
   and its documentation for any purpose, without fee, and
   without written agreement is hereby granted, provided that the
   above copyright notice and the following two paragraphs appear
   in all copies of this software.

   IN NO EVENT SHALL THE UNIVERSITY OF CALIFORNIA BE LIABLE TO
   ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR
   CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE
   AND ITS DOCUMENTATION, EVEN IF THE UNIVERSITY OF CALIFORNIA
   HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

   THE UNIVERSITY OF CALIFORNIA SPECIFICALLY DISCLAIMS ANY
   WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED
   WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
   PURPOSE.  THE SOFTWARE PROVIDED HEREUNDER IS ON AN "AS IS"
   BASIS, AND THE UNIVERSITY OF CALIFORNIA HAS NO OBLIGATION TO
   PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR
   MODIFICATIONS.
*/

#include "threads/synch.h"
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
#include "threads/interrupt.h"
#include "threads/thread.h"
#include "list.h"

/* Initializes semaphore SEMA to VALUE.  A semaphore is a
   nonnegative integer along with two atomic operators for
   manipulating it:

   - down or "P": wait for the value to become positive, then
     decrement it.

   - up or "V": increment the value (and wake up one waiting
     thread, if any). */
static bool comparator_greater_lock_priority(const struct list_elem*
a, const struct list_elem* b, void* aux UNUSED);
static bool
comparator_greater_thread_priority(
  const struct list_elem* a,
  const struct list_elem* b, void* aux UNUSED);
static bool comparator_greater_sema_priority(struct list_elem*  x,
struct list_elem* y, void* aux UNUSED);

void
```

```
sema_init (struct semaphore *sema, unsigned value)
{
  ASSERT (sema != NULL);

  sema->value = value;
  list_init (&sema->waiters);
}

/* Down or "P" operation on a semaphore.  Waits for SEMA's value
   to become positive and then atomically decrements it.

   This function may sleep, so it must not be called within an
   interrupt handler.  This function may be called with
   interrupts disabled, but if it sleeps then the next scheduled
   thread will probably turn interrupts back on. */
void
sema_down (struct semaphore *sema)
{
  enum intr_level old_level;

  ASSERT (sema != NULL);
  ASSERT (!intr_context ());

  old_level = intr_disable ();
  while (sema->value == 0)
    {
      list_insert_ordered(&sema->waiters,      &thread_current()->elem,
comparator_greater_thread_priority,  NULL);  /*  вставка  по  убыванию
приоритета в лист ожидающих */
      thread_block ();
    }
  sema->value--;
  intr_set_level (old_level);
}

/* Down or "P" operation on a semaphore, but only if the
   semaphore is not already 0.  Returns true if the semaphore is
   decremented, false otherwise.

   This function may be called from an interrupt handler. */
bool
sema_try_down (struct semaphore *sema)
{
  enum intr_level old_level;
  bool success;

  ASSERT (sema != NULL);

  old_level = intr_disable ();
  if (sema->value > 0)
    {
      sema->value--;
      success = true;
    }
  else
    success = false;
  intr_set_level (old_level);
```

```
  return success;
}

/* Up or "V" operation on a semaphore.  Increments SEMA's value
   and wakes up one thread of those waiting for SEMA, if any.

   This function may be called from an interrupt handler. */
void
sema_up (struct semaphore *sema)
{
  enum intr_level old_level;
  struct thread* target = NULL;

  ASSERT (sema != NULL);

  old_level = intr_disable ();

  sema->value++;

  if (!list_empty (&sema->waiters))
  {
    list_sort(&(sema-
>waiters),comparator_greater_thread_priority,NULL);

    target = list_entry(list_pop_front(&sema->waiters),struct thread,
elem);
    thread_unblock(target);
  }

  intr_set_level (old_level);
}

static void sema_test_helper (void *sema_);

/* Self-test for semaphores that makes control "ping-pong"
   between a pair of threads.  Insert calls to printf() to see
   what's going on. */
void
sema_self_test (void)
{
  struct semaphore sema[2];
  int i;

  printf ("Testing semaphores...");
  sema_init (&sema[0], 0);
  sema_init (&sema[1], 0);
  thread_create ("sema-test", PRI_DEFAULT, sema_test_helper, &sema);
  for (i = 0; i < 10; i++)
    {
      sema_up (&sema[0]);
      sema_down (&sema[1]);
    }
  printf ("done.\n");
}

/* Thread function used by sema_self_test(). */
static void
sema_test_helper (void *sema_)
```

```
{
  struct semaphore *sema = sema_;
  int i;

  for (i = 0; i < 10; i++)
    {
      sema_down (&sema[0]);
      sema_up (&sema[1]);
    }
}
/* Initializes LOCK.  A lock can be held by at most a single
   thread at any given time.  Our locks are not "recursive", that
   is, it is an error for the thread currently holding a lock to
   try to acquire that lock.

   A lock is a specialization of a semaphore with an initial
   value of 1.  The difference between a lock and such a
   semaphore is twofold.  First, a semaphore can have a value
   greater than 1, but a lock can only be owned by a single
   thread at a time.  Second, a semaphore does not have an owner,
   meaning that one thread can "down" the semaphore and then
   another one "up" it, but with a lock the same thread must both
   acquire and release it.  When these restrictions prove
   onerous, it's a good sign that a semaphore should be used,
   instead of a lock. */
void
lock_init (struct lock *lock)
{
  ASSERT (lock != NULL);

  lock->holder = NULL;
  lock->priority = PRI_MIN;
  sema_init (&lock->semaphore, 1);
}

/* Acquires LOCK, sleeping until it becomes available if
   necessary.  The lock must not already be held by the current
   thread.

   This function may sleep, so it must not be called within an
   interrupt handler.  This function may be called with
   interrupts disabled, but interrupts will be turned back on if
   we need to sleep. */
void
lock_acquire (struct lock *lock)
{
  ASSERT (lock != NULL);
  ASSERT (!intr_context ());
  ASSERT (!lock_held_by_current_thread (lock));

  // priority donation, when locking
  struct lock* current_lock=lock;
  struct thread* t_holder = lock->holder; // current holder thread
  struct thread* t_current = thread_current();

  // the current process is waiting on [lock]
  t_current->waiting_lock=lock;
```

```
  if(t_holder == NULL)
  {
    current_lock->priority = t_current->priority;
  }

  while(t_holder !=NULL && t_holder->priority < t_current->priority)
  {
    //donate priority to [t_holder]
    thread_priority_donate(t_holder,t_current->priority);

    if (current_lock->priority < t_current->priority)
    {
      current_lock->priority = t_current -> priority;
    }

    current_lock = t_holder->waiting_lock;
    if (current_lock == NULL) break;
    t_holder = current_lock->holder;


  }

  sema_down (&lock->semaphore);
  lock->holder = thread_current ();

  lock->holder->waiting_lock = NULL; // no longer waiting
  list_insert_ordered(&(lock->holder->locks),        &(lock->lockelem),
comparator_greater_lock_priority, NULL);
}

/* Tries to acquires LOCK and returns true if successful or false
   on failure.  The lock must not already be held by the current
   thread.

   This function will not sleep, so it may be called within an
   interrupt handler. */
bool
lock_try_acquire (struct lock *lock)
{
  bool success;

  ASSERT (lock != NULL);
  ASSERT (!lock_held_by_current_thread (lock));

  success = sema_try_down (&lock->semaphore);
  if (success)
    lock->holder = thread_current ();
  return success;
}

/* Releases LOCK, which must be owned by the current thread.

   An interrupt handler cannot acquire a lock, so it does not
   make sense to try to release a lock within an interrupt
   handler. */
void
lock_release (struct lock *lock)
{
```

```
  ASSERT (lock != NULL);
  ASSERT (lock_held_by_current_thread (lock));

  struct thread* t_current = thread_current();

  lock->holder = NULL;
  sema_up (&lock->semaphore);

  // Remove the lock from locklist
  list_remove(&lock->lockelem);

  // priority donation: restoration
  if(list_empty(&t_current->locks))
  {
    // no more locks: there is no priority donors
    // the original priority of the current thread
    thread_priority_donate(t_current, t_current->original_priority);
  }
  else
  {
    // donated: lookup the donors, find the highest priority lock
    // then it should be the (newly updated) donated priority of t
    list_sort(&(t_current-
>locks),comparator_greater_lock_priority,NULL);
    struct  lock*  highest_lock  =  list_entry(list_front(&(t_current-
>locks)),struct lock, lockelem);
    thread_priority_donate(t_current, highest_lock->priority);
  }
}

/* Returns true if the current thread holds LOCK, false
   otherwise.  (Note that testing whether some other thread holds
   a lock would be racy.) */
bool
lock_held_by_current_thread (const struct lock *lock)
{
  ASSERT (lock != NULL);

  return lock->holder == thread_current ();
}
/* One semaphore in a list. */
struct semaphore_elem
  {

    struct list_elem elem;              /* List element. */
    struct semaphore semaphore;         /* This semaphore. */
  };

/* Initializes condition variable COND.  A condition variable
   allows one piece of code to signal a condition and cooperating
   code to receive the signal and act upon it. */
void
cond_init (struct condition *cond)
{
  ASSERT (cond != NULL);

  list_init (&cond->waiters);
}
```

```
/* Atomically releases LOCK and waits for COND to be signaled by
   some other piece of code.  After COND is signaled, LOCK is
   reacquired before returning.  LOCK must be held before calling
   this function.

   The monitor implemented by this function is "Mesa" style, not
   "Hoare" style, that is, sending and receiving a signal are not
   an atomic operation.  Thus, typically the caller must recheck
   the condition after the wait completes and, if necessary, wait
   again.

   A given condition variable is associated with only a single
   lock, but one lock may be associated with any number of
   condition variables.  That is, there is a one-to-many mapping
   from locks to condition variables.

   This function may sleep, so it must not be called within an
   interrupt handler.  This function may be called with
   interrupts disabled, but interrupts will be turned back on if
   we need to sleep. */
void
cond_wait (struct condition *cond, struct lock *lock)
{
  struct semaphore_elem waiter;

  ASSERT (cond != NULL);
  ASSERT (lock != NULL);
  ASSERT (!intr_context ());
  ASSERT (lock_held_by_current_thread (lock));

  sema_init (&waiter.semaphore, 0);

  waiter.semaphore.priority = thread_current()->priority;

  list_insert_ordered(&cond->waiters,                    &(waiter.elem),
comparator_greater_sema_priority,  NULL);  //  вставка  в  лист  ожидания
условия по убыванию приоритета

  lock_release (lock);
  sema_down (&waiter.semaphore);
  lock_acquire (lock);
}

/* If any threads are waiting on COND (protected by LOCK), then
   this function signals one of them to wake up from its wait.
   LOCK must be held before calling this function.

   An interrupt handler cannot acquire a lock, so it does not
   make sense to try to signal a condition variable within an
   interrupt handler. */
void
cond_signal (struct condition *cond, struct lock *lock UNUSED)
{
  ASSERT (cond != NULL);
  ASSERT (lock != NULL);
  ASSERT (!intr_context ());
  ASSERT (lock_held_by_current_thread (lock));
```

```
  if (!list_empty (&cond->waiters))
    sema_up (&list_entry (list_pop_front (&cond->waiters),
                          struct semaphore_elem, elem)->semaphore);
}

/* Wakes up all threads, if any, waiting on COND (protected by
   LOCK).  LOCK must be held before calling this function.

   An interrupt handler cannot acquire a lock, so it does not
   make sense to try to signal a condition variable within an
   interrupt handler. */
void
cond_broadcast (struct condition *cond, struct lock *lock)
{
  ASSERT (cond != NULL);
  ASSERT (lock != NULL);

  while (!list_empty (&cond->waiters))
    cond_signal (cond, lock);
}

static bool
comparator_greater_thread_priority(
  const struct list_elem* a,
  const struct list_elem* b, void* aux UNUSED)
{
  const struct thread* x = list_entry(a, struct thread, elem);
  const struct thread* y = list_entry(b, struct thread, elem);
  ASSERT(x != NULL && y != NULL);
  return x->priority > y->priority;
}

static bool comparator_greater_lock_priority(const struct list_elem*
a, const struct list_elem* b, void* aux UNUSED)
{
  const struct lock* x = list_entry(a, struct lock, lockelem);
  const struct lock* y = list_entry(b, struct lock, lockelem);
  ASSERT(x != NULL && y !=NULL);
  return x->priority > y->priority;
}

static bool comparator_greater_sema_priority(struct list_elem* a,
struct list_elem* b, void* aux UNUSED)
{
  struct semaphore_elem*  x= list_entry(a,struct semaphore_elem,elem);
  struct semaphore_elem* y= list_entry(b,struct semaphore_elem,elem);
  ASSERT(x!=NULL && y !=NULL);
  return x->semaphore.priority > y->semaphore.priority;

}
```